

Background

The following may be helpful when reading or debugginfg this notebook:

Standard Amino Acids

The FASTA data format represents each amino acid with a single character as follows:

1-Letter Code	3-Letter Code	Amino Acid Name
A	Ala	Alanine
C	Cys	Cysteine
D	Asp	Aspartic acid
E	Glu	Glutamic acid
F	Phe	Phenylalanine
G	Gly	Glycine
H	His	Histidine
I	Ile	Isoleucine
K	Lys	Lysine
L	Leu	Leucine
M	Met	Methionine
N	Asn	Asparagine
P	Pro	Proline
Q	Gln	Glutamine
R	Arg	Arginine

1-Letter Code	3-Letter Code	Amino Acid Name
S	Ser	Serine
T	Thr	Threonine
V	Val	Valine
W	Trp	Tryptophan
Y	Tyr	Tyrosine

Ambiguous & Special Characters

The FASTA format also uses the following abbreviations for ambiguous amino acid identification and special characters:

1-Letter Code	Description / Meaning
B	Aspartic acid (D) or Asparagine (N)
J	Leucine (L) or Isoleucine (I)
X	Unknown or any amino acid
Z	Glutamic acid (E) or Glutamine (Q)
*	Translation stop codon
-	Gap of missing or unsequenced amino acid

Environment Setup

In this section we're setting up the environment for the rest of the notebook.

```
In [2]: # Core libraries
import pandas as pd
import numpy as np
import warnings
import random
```

```
import torch
import esm
import re

# File read / write
import pickle

# Machine Learning / Modeling
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import (f1_score, roc_curve,
                             confusion_matrix, ConfusionMatrixDisplay, roc_auc_score,
                             average_precision_score, precision_recall_curve)

# Gradient boosting libraries

# Deep Learning
from tensorflow import keras
from tensorflow.keras import layers

# Statistical / other utilities

# Explainability

# Visualization
import matplotlib.pyplot as plt

# BioPython
from Bio import SeqIO

# Environment Settings
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
np.random.seed(42)
warnings.filterwarnings('ignore')

# Filepaths for source data files
path1 = '../data/raw/uniprotkb_proteome_UP000002311.fasta'
path2 = '../data/raw/BIOGRID-ORGANISM-Saccharomyces_cerevisiae_S288c-5.0.259.mitab.txt'
```

```
/Users/austinsoelwin/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3
v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.c
om/urllib3/urllib3/issues/3020
warnings.warn(
```

In [3]: *# New installations required for this project*

```
#%pip install biopython
#pip install torch
#pip install safetensors==0.4.5
#pip install fair-esm
```

Helper Functions

In this section we're defining some helper functions for use in the rest of the notebook.

```
In [3]: def read_fasta(file_path):
        """Funtion to read the ID and Sequence info from the FASTA file"""
        sequences = {}
        current_id = None
        current_seq = []

        with open(file_path, "r") as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue

                # Lines starting with '>' indicate a new sequence ID
                if line.startswith(">"):
                    if current_id:
                        sequences[current_id] = "".join(current_seq)
                    current_id = line[1:] # Remove the '>' character
                    current_seq = []
                else:
                    current_seq.append(line)

        # Add the final sequence from the file
```

```
        if current_id:
            sequences[current_id] = "".join(current_seq)

    return sequences

def extract_locuslink(s):
    """Function to extract the Uniprot gene IDs from the Biogrid dataframe"""
    match = re.search(r'locuslink:([^\|]+)', s)
    return match.group(1) if match else None

def get_interactors(bg, protein_id):
    """Function to extract interactions from Biogrid dataframe"""
    # Protein appears in column A
    a_partners = bg.loc[
        bg['Gene_A'] == protein_id,
        'Gene_B'
    ]

    # Protein appears in column B
    b_partners = bg.loc[
        bg['Gene_B'] == protein_id,
        'Gene_A'
    ]

    # Combine, deduplicate, and return as a list
    return pd.concat([a_partners, b_partners]).drop_duplicates().tolist()

def get_esm_embedding(model, sequence, batch_converter):
    """Function to create the embedding for a given protein"""
    batch_labels, batch_strs, batch_tokens = batch_converter([("P", sequence)])
    batch_tokens = batch_tokens.to(device)
    with torch.no_grad():
        results = model(batch_tokens, repr_layers=[last_layer], return_contacts=False)
    token_representations = results["representations"][last_layer]
    protein_representations = token_representations[0,1:-1,:].mean(dim=0).cpu().numpy()
    return protein_representations
```

```
def random_protein(length):  
    """Function to generate a random amino acid sequence of fixed length"""  
    rand01 = ''.join(random.choices("ACDEFGHIKLMNPQRSTVWY", k=length))  
    return rand01  
  
def get_pairwise_features(embA, embB):  
    """Function to create the pair-wise feature set for 2 embeddings"""  
    pairwise = np.concatenate([embA, embB, np.abs(embA - embB), embA * embB])  
    return pairwise  
  
def create_balanced_dataset(interaction_matrix, embeddings, n=1000, random_state=42):  
    """This function returns a balanced dataset (equal numbers of interacting and non-interacting pairs)  
    of pairwise embeddings of specified size for training the NN  
    """  
    rng = np.random.default_rng(random_state)  
  
    proteins = interaction_matrix.index.to_numpy()  
  
    # Get upper-triangle indices, excluding diagonal  
    i, j = np.triu_indices(len(proteins), k=1)  
  
    # Get corresponding interaction values  
    values = interaction_matrix.values[i, j]  
  
    # Separate positive and negative pairs  
    positive_idx = np.where(values == 1)[0]  
    negative_idx = np.where(values == 0)[0]  
  
    # Check that enough pairs exist  
    if len(positive_idx) < n:  
        raise ValueError(  
            f"Only {len(positive_idx)} positive pairs available; "  
            f"cannot sample {n}."  
        )  
  
    if len(negative_idx) < n:  
        raise ValueError(  
            f"Only {len(negative_idx)} negative pairs available; "  
            f"cannot sample {n}."  
        )
```

```
# Randomly select n of each
positive_sample = rng.choice(
    positive_idx, size=n, replace=False
)

negative_sample = rng.choice(
    negative_idx, size=n, replace=False
)

selected = np.concatenate([
    positive_sample,
    negative_sample
])

# Shuffle positive and negative examples together
rng.shuffle(selected)

X = []
y = []
pairs = []

for idx in selected:

    protein_a = proteins[i[idx]]
    protein_b = proteins[j[idx]]

    emb_a = embeddings[protein_a]
    emb_b = embeddings[protein_b]

    # Symmetric pair representation
    pair_embedding = np.concatenate([
        np.abs(emb_a - emb_b),
        emb_a * emb_b
    ]) # Could use alternate function "get_pairwise_features(emb1, emb2)" here

    X.append(pair_embedding)
    y.append(values[idx])
    pairs.append((protein_a, protein_b))
```

```
return np.array(X), np.array(y), pairs
```

Section 1 - Data Import

In this section we are importing amino acid sequences for all 6,067 amino acids in *S. cerevisiae*, and interaction data from BioGrid.

```
In [4]: # Import dataset
# Print 3 First Rows
for i, record in enumerate(SeqIO.parse(path1, "fasta")):
    if i >= 3:
        break
    print(f"ID: {record.id}")
    print(f"Sequence: {record.seq}")
    print(f"Length: {len(record.seq)}\n")
```

ID: sp|A0A0B7P3V8|YP41B_YEAST

Sequence: MATPVRDETRNVIDDNISARIQSKVKTNDTVRQTPSSLRKVSIDKDEQVKQYQRNLNRFKTI LNGLKAEKKLSETDDIQMLAEKLLKLGETIDKVENRIVDLVEKI
 QLLETNNENNILHEHIDATGYTTLFDLTSTNKRFPKDCVFDYRTNNVENIPILLNNFKKFIKKYQFDDVFENDIIEIDPRENEILCKIIKEGLGESLDIMNTNTTDFRIIDGL
 KNKYRSLHGRDVRIRAWEKVLVDTCRNSALLMNKLQKLVLMEKWIFSKCCQDCPNLKDYLQEAIMGTLHESLRNSVKQRLYNIPHNVGINHEEFINTVIETVIDLSPIADDQIE
 NSCMYCKSVFHCINCKKKPNRELRPDSTNFSKTYLQGAQRQQQLKSSAKEQKSWNKTQKKS NKVYNSKKLVIIDTSGSVNITNDKTLHNYEDSNRSTRFFGIGKNSSSVKGY
 GYIKIKNGHNNTDNKCLLTYYVPEEESTIISCYDLAKKTKMVL SRKYTRLGNKI IKTIVNGVIHVKMNELIERPSDDSKINAIKPTSSPGFKLNKRSITLED AHKRMGHTGIQ
 QIENSIKHNHYEESLDLIKEPNEFWCQTKISKATKRNHYT GSMNNHSTDHEPGSSWCMDIFGPVSSSNADTKRYMLIMVDNNTRYCMTSTHFNKNAETILAQIRKNIQYVETQFD
 RKVREINSDRGTEFTNDQIEEYFISKGIHHILTSTQDHAANGRAERYIRTIVDATTLLRQSNLRVKFWEYAVTSATNIRNCLEHKSTGKLPLKAISRQPVTVRLMSFLPFGEKGI
 IWNHNHKKLKPSGLPSIILCKDPNSYGYKFFIPSKNKIVTSDNYTIPNYTMDGRVRNTQNIYKSHQFSSHNDNEEDQIETVTNLCEALENYEDDNKPITRLEDLFTEEELSQIDSN
 AKYPPSPNNLEGLDYVFSDEESGDYDVESELSTNTSISTDKNKILSNKDFNSELASTEISISEIDKKGLINTSHIDEDKYDEKVRHIPSIIQEKLVGSKNTIKINDENRISDR
 IRSKNIGSILNTGLSRCVDITDESITNKDESMHNAKPELIQE QFNKTNHETSFPKEGSIGTKCKIPKYRQ

Length: 1104

ID: sp|D6VTK4|STE2_YEAST

Sequence: MSDAAPSLSNLFYDPTYNPGQSTINYTSIYNGGSTITFDELQGLVNSTVTQAIMFGVRCGAAALTLIVMWMTSRSRKTPIFIINQVSLFLIILHSALYFKYLLSNY
 SSVTYALTGFPPQFISRGDVHVGATNIIQVLLVASIETSLVFQIKVIFTGDNFKRIGLMLTSISFTLGIATVTMYFVSAVKGMIVTYNDVSATQDKYFNASTILLASSINFMSFVL
 VVKLILAIRSRRFLGLKQFDSFHILLIMSCQSLLVPSIIFILAYSLKPNQGTVDLTTVATLLAVLSLPLSSMWATAANNASKTNTITSDFTTSTDRFYPGTLSSFQTD SINNDAKS
 SLRSRLYDLYPRRKETTS DKHSERTFVSETADDIEKNQFYQLPTPTSSKNTRIGPFADASYKEGEVEPVDMYTPDTAADEEARKFWTEDNNNL

Length: 431

ID: sp|013297|CET1_YEAST

Sequence: MSYTDNPPQTKRALSLDDL VNH DENEKVKLQKLSEAANGSRPFAENLESDINQTETGQAAPIDNYKESTGHGSHSQPKSRKSSNDDEETD TDD EMGASGEINFDS
 EMDFDYDKQHRNLLSNGSPPMNDGSDANAKLEKPSDDSIHQNSKDEEQRI PKQGNENIASNYITQVPLQKQKQTEKKIAGNAVGSVVKKEEANA AAVDNI FEEKATLQSKKNNI
 KRDLLEV LNEISASSKPSKYRNVPIWAQKWKPTIKALQSINVKDLKIDPSFLNIIPDDDLTKSVQDWYATIYSIAPELRSFIELEMKFGVIIDAKGPDRVNPVSSQCVFTELD A
 LTPNIDASLFKELSKYIRGISEVTENTGKFSIIESQTRDSVYRVGLSTQRPRFLRMSTDIKTGRVGQFIEKRHVAQLLLYSPKDSYDVKISLNLELPVPDNDPPEKYKSQSPISER
 TKDRVSYIHNDSCTRIDITKVENHNQNSKSRQSETTHEVELEINTPALLNAFDNITNDSKEYASLIRTFLLNGTIIRKLSLSYEIFEGSKKVM

Length: 549

```
In [5]: # Create the protein-sequence dictionary and a dictionary of protein residue length
sequence_dictionary = {}
length_dictionary = {}
fasta_data = read_fasta(path1)

for seq_id, sequence, in fasta_data.items():
    match = re.search(r'GN=(\S+)', seq_id)
    if match:
        gene_name = match.group(1)
        sequence_dictionary[gene_name] = sequence
        length_dictionary[gene_name] = len(sequence)
```

```
# Create a list of all proteins for future use
gene_list = list(sequence_dictionary.keys())
```

```
In [6]: # Inspect the results
print(gene_list[:5])
print(len(gene_list))
```

```
['TY4B-P', 'STE2', 'CET1', 'FOB1', 'RPS9A']
6067
```

```
In [7]: # Check if a particular gene is included in the Uniprot data
if 'CDC73' in gene_list:
    print("Item found!")
```

Item found!

Here we are importing the interaction data from BioGrid.

```
In [8]: # Now extract the Biogrid data and create the Biogrid dataframe
bg = pd.read_csv(path2,
    sep="\t",
    header=None,
    dtype=str
)

print(bg.shape)

# Reset the row and column indices
bg.columns = bg.iloc[0]
bg = bg[1:]
bg.columns.name = None
bg = bg.reset_index(drop=True)

# Inspect the results
print(bg.shape)
```

```
(885690, 15)
(885689, 15)
```

```
In [9]: # Check the Biogrid data at a particular dataframe location
print(bg.at[1, 'Alt IDs Interactor A'])
```

biogrid:34272|entrez gene/locuslink:CTR9|entrez gene/locuslink:Y0L145C|uniprot/swiss-prot:P89105|refseq:NP_014496

```
In [10]: # Extract the interactor A and B genes and create a new column for each
bg['Gene_A'] = bg['Alt IDs Interactor A'].apply(extract_locuslink)
bg['Gene_B'] = bg['Alt IDs Interactor B'].apply(extract_locuslink)

# Inspect the results
bg.head()
```

Out[10]:

	#ID Interactor A	ID Interactor B	Alt IDs Interactor A	Alt IDs Interactor B	Aliases Interactor A
0	entrez gene/locuslink:851136	entrez gene/locuslink:854020	biogrid:31676 entrez gene/locuslink:CDC73 ent...	biogrid:34272 entrez gene/locuslink:CTR9 entre...	entrez gene/locuslink:L000002792(gene name syn..
1	entrez gene/locuslink:854020	entrez gene/locuslink:851136	biogrid:34272 entrez gene/locuslink:CTR9 entre...	biogrid:31676 entrez gene/locuslink:CDC73 ent...	entrez gene/locuslink:CDP1(gene name synonym) ..
2	entrez gene/locuslink:851136	entrez gene/locuslink:854290	biogrid:31676 entrez gene/locuslink:CDC73 ent...	biogrid:34518 entrez gene/locuslink:LEO1 entre...	entrez gene/locuslink:L000002792(gene name syn..
3	entrez gene/locuslink:854290	entrez gene/locuslink:851136	biogrid:34518 entrez gene/locuslink:LEO1 entre...	biogrid:31676 entrez gene/locuslink:CDC73 ent...	entrez gene/locuslink:L000000936(gene name syn..
4	entrez gene/locuslink:851136	entrez gene/locuslink:852582	biogrid:31676 entrez gene/locuslink:CDC73 ent...	biogrid:32973 entrez gene/locuslink:PAF1 entre...	entrez gene/locuslink:L000002792(gene name syn..

This cell takes quite a while to run locally; therefore writing the results to file for future use.

```
In [ ]: # CELL_TAKE_TIME
# Comment out the following code if you have already created the interactions.pkl file and don't want to re-run thi
# Now we make the interaction dictionary and write to file for future use
# ia_dic = {}

# # Iterate through each gene and extract interactions
# for gene in gene_list:
#     interactors = get_interactors(bg, gene)
#     ia_dic[gene] = interactors

# # Write the interaction dictionary to file
# with open('../data/processed/interactions.pkl', 'wb') as file:
#     pickle.dump(ia_dic, file)
```

```
In [ ]: # Read the interaction dictionary from the binary file
with open('../data/processed/interactions.pkl', 'rb') as file:
    interaction_dictionary = pickle.load(file)
```

```
In [ ]: # Inspect the results (first 5 keys, first 5 values each)
{k: v[:5] for k, v in list(interaction_dictionary.items())[:5]}
```

Section 2 - Data Cleaning & Pre-processing

In this section we perform some data cleaning and pre-processing operations.

```
In [20]: #First, we need to make sure we have sequence and interaction data for all proteins.

# Find entries in the interaction dictionary that are empty
empty_keys = [key for key, value in interaction_dictionary.items() if not value]
print('Number of empty entries in the interaction dictionary:')
print(len(empty_keys))

# Remove them from the sequence and interaction dictionaries
for key in empty_keys:
```

```

sequence_dictionary.pop(key, None)
length_dictionary.pop(key, None)
interaction_dictionary.pop(key, None)

print('')
print('Number of proteins for which he have sequence AND interaction data:')
print(len(sequence_dictionary))

new_gene_list = list(sequence_dictionary.keys())

```

Number of empty entries in the interaction dictionary:
397

Number of proteins for which he have sequence AND interaction data:
5670

```

In [21]: # Next we need to remove any interactors for which we don't have sequence data
interaction_dictionary_filtered = {
    protein: [p for p in partners if p in new_gene_list]
    for protein, partners in interaction_dictionary.items()
}

```

```

In [22]: # Inspect the results (first 5 keys, first 5 values each)
{k: v[:5] for k, v in list(interaction_dictionary_filtered.items())[:5]}

```

```

Out[22]: {'STE2': ['STE2', 'STE4', 'GPA1', 'STE5', 'AXL1'],
          'CET1': ['TUP1', 'CEG1', 'CET1', 'SPT4', 'SPT5'],
          'FOB1': ['NET1', 'SIR2', 'CDC14', 'FOB1', 'NSI1'],
          'RPS9A': ['MPP10', 'RPS9B', 'RTC6', 'MOB2', 'HIR1'],
          'COQ4': ['NDI1', 'NDE1', 'COQ9', 'COQ8', 'GLE1']}

```

```

In [23]: # Finally we create the overall interaction matrix using the interaction dictionary - a square
# matrix 5670 x 5670 with 1's indicating interactions, 0's indicating no interaction.
# Note that this will be a symmetric matrix.

# We will call this the 'Master Interaction Matrix' or MIM
MIM = pd.DataFrame(0, index=new_gene_list, columns=new_gene_list)

for protein, partners in interaction_dictionary_filtered.items():
    MIM.loc[protein, partners] = 1

```

```
In [25]: # Perform some checks to make sure MIM is setup correctly, check sparsity
row = 'CET1'
column = 'STE2'

print(f'MIM entry at {row}, {column}:')
print(MIM.loc[row, column])
print('')

# See how sparse MIM
count_ones = np.count_nonzero(MIM)
print('Number of non-zero entries in the MIM:')
print(count_ones)
print('')

print('Total number of entries in the MIM:')
num_entries = 5670*5670
print(num_entries)
print('')

print('Sparsity of the MIM:')
print(count_ones/num_entries)
```

MIM entry at CET1, STE2:
0

Number of non-zero entries in the MIM:
1220487

Total number of entries in the MIM:
32148900

Sparsity of the MIM:
0.037963569515597734

Section 3 - Create Embeddings

In this section we are creating the embeddings for all the proteins for which we have sequence AND interaction data. We will then perform some checks to make sure the embeddings are behaving as expected, by comparing the cosine similarity between various

embeddings.

```
In [26]: # Load a pre-trained ESM embedding model for use in the create embedding function

# NOTE: The code in this cell was derived from the following ESM2-Tutorial repository:
# https://github.com/ProteinVision/ESM2-Tutorial/blob/main/ESM2.ipynb

# Utilize GPU resources if available, otherwise default to CPU
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Using device {device} for data processing.')

# Load the model producing 1280-dim embeddings
model, alphabet = esm.pretrained.esm2_t33_650M_UR50D()
last_layer = len(model.layers)
batch_converter = alphabet.get_batch_converter()
model.eval()
model.to(device)

# Print the token alphabet (first 5 items)
dict(list(alphabet.to_dict().items())[:5])
```

Using device cpu for data processing.

```
Out[26]: {'<cls>': 0, '<pad>': 1, '<eos>': 2, '<unk>': 3, 'L': 4}
```

```
In [27]: # Perform some basic checks of the embedding

# Define a sequence and confirm the embedding has the right dimensionality
seq1 = 'MSDAAPSLSNLFYDPTYNPGQSTINYTSIYGNGSTITFDELQGLVNSTVTQAIMFGVRCGAAALTLIVMWMTSRSRKTIPIFIINQVSLFLIILHSALYFKYLLSNYS'
emb1 = get_esm_embedding(model, seq1, batch_converter)

# Confirm shape, content
print('Length of sequence embedding:')
print(len(emb1))
print('')
print('Values in embedding:')
print(emb1)
```

Length of sequence embedding:
1280

Values in embedding:
[-0.04797408 -0.00718806 0.02167165 ... -0.14549576 0.04699722
0.07034333]

```
In [28]: # Perform some more advanced checks

# Define two similar proteins - human alpha globin (HAG) and human beta globin (HBG)
HAG = "MVLSPADKTNVKAAGWKVGAHAGEYGAEALERMFSLFPTTKTYFPHFDLSHGSAQVKGHGKKVADALTNAVAHVDDMPNALSALSDLHAHKLRVDPVNFKLLSHCLLV
HBG = "MVHLTPEEKSAVTALWGKVNVDEVGGEALGRLLVVYPWTQRFFESFGDLSTPDAMGNPKVKAHGKKVLGAFSDGLAHLNKLKGTATLSELHCDKLHVDPENFRLLG

# Pick a random S. cerevisiae protein
CDC73 = sequence_dictionary['CDC73']

# Create 2 random proteins of similar length
rand1 = random_protein(150)
rand2 = random_protein(150)

# Check the cosine similarity between the two
alpha = get_esm_embedding(model, HAG, batch_converter)
beta = get_esm_embedding(model, HBG, batch_converter)
delta = get_esm_embedding(model, CDC73, batch_converter)
gamma1 = get_esm_embedding(model, rand1, batch_converter)
gamma2 = get_esm_embedding(model, rand2, batch_converter)
```

```
In [29]: # Calculate the cosine similarity between HAG and HBG
print("Cosine similarity between human alpha globin (HAG) and human beta globin (HBG) embeddings:")
a = alpha
b = beta
similarity = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(similarity)
print('')

# Calculate the cosine similarity between HAG and a protein sequence from S. cerevisiae
print("Cosine similarity between human alpha globin (HAG) and CDC73:")
a = alpha
b = delta
similarity = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

```

print(similarity)
print('')

# Calculate the cosine similarity between HAG and a random protein sequence
print("Cosine similarity between human alpha globin (HAG) and a random protein sequence:")
a = alpha
b = gamma1
similarity = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(similarity)
print('')

# Calculate the cosine similarity between HAG and the other random protein sequence
print("Cosine similarity between human alpha globin (HAG) and another random protein sequence:")
a = alpha
b = gamma2
similarity = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(similarity)

```

Cosine similarity between human alpha globin (HAG) and human beta globin (HBG) embeddings:
0.95145655

Cosine similarity between human alpha globin (HAG) and CDC73:
0.8252415

Cosine similarity between human alpha globin (HAG) and a random protein sequence:
0.6576298

Cosine similarity between human alpha globin (HAG) and another random protein sequence:
0.6556077

This cell takes quite a while to run locally; therefore writing the results to file for future use.

```

In [30]: # CELL_TAKE_TIME
# Comment out the following code if you have already created the embeddings.pkl file and don't want to re-run this
# Create a dictionary of embeddings for all proteins, and write to file
# embedding_dict = {}

# # Loop through each protein in the set
# for gene in new_gene_list:
#     sequence = sequence_dictionary[gene]
#     embedding = get_esm_embedding(model, sequence, batch_converter)

```

```
#     embedding_dict[gene] = embedding

# # Write the embeddings dictionary to file
# with open('../data/processed/embeddings.pkl', 'wb') as file:
#     pickle.dump(embedding_dict, file)
```

```
In [31]: # Read the embedding dictionary from the binary file
with open('../data/processed/embeddings.pkl', 'rb') as file:
    embedding_dict = pickle.load(file)
```

```
In [32]: # Inspect the results (first 5 keys, first 5 values each)
{k: v[:5] for k, v in list(embedding_dict.items())[:5]}
```

```
Out[32]: {'STE2': array([-0.04797408, -0.00718806,  0.02167165,  0.06603702, -0.03343712],
      dtype=float32),
      'CET1': array([-0.00223311, -0.01582935,  0.04741028,  0.01531323,  0.0022517 ],
      dtype=float32),
      'FOB1': array([ 0.01796917, -0.07855558,  0.03557299,  0.01860837,  0.02187089],
      dtype=float32),
      'RPS9A': array([ 0.01126087, -0.01193911,  0.04757049,  0.01326585, -0.06809363],
      dtype=float32),
      'COQ4': array([-0.0266132 ,  0.03327498, -0.02508251, -0.03675416,  0.01357298],
      dtype=float32)}
```

```
In [33]: # Create a second embedding model using the smaller pre-trained ESM2

# NOTE: The code in this cell was derived from the following ESM2-Tutorial repository:
# https://github.com/ProteinVision/ESM2-Tutorial/blob/main/ESM2.ipynb

# Utilize GPU resources if available, otherwise default to CPU
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Using device {device} for data processing.')

# Load the model producing 640-dim embeddings
model_small, alphabet_small = esm.pretrained.esm2_t6_8M_UR50D()
last_layer = len(model_small.layers)
batch_converter = alphabet_small.get_batch_converter()
model_small.eval()
model_small.to(device)
```

Using device cpu for data processing.

```
Out[33]: ESM2(
  (embed_tokens): Embedding(33, 320, padding_idx=1)
  (layers): ModuleList(
    (0-5): 6 x TransformerLayer(
      (self_attn): MultiheadAttention(
        (k_proj): Linear(in_features=320, out_features=320, bias=True)
        (v_proj): Linear(in_features=320, out_features=320, bias=True)
        (q_proj): Linear(in_features=320, out_features=320, bias=True)
        (out_proj): Linear(in_features=320, out_features=320, bias=True)
        (rot_emb): RotaryEmbedding()
      )
      (self_attn_layer_norm): LayerNorm((320,), eps=1e-05, elementwise_affine=True)
      (fc1): Linear(in_features=320, out_features=1280, bias=True)
      (fc2): Linear(in_features=1280, out_features=320, bias=True)
      (final_layer_norm): LayerNorm((320,), eps=1e-05, elementwise_affine=True)
    )
  )
  (contact_head): ContactPredictionHead(
    (regression): Linear(in_features=120, out_features=1, bias=True)
    (activation): Sigmoid()
  )
  (emb_layer_norm_after): LayerNorm((320,), eps=1e-05, elementwise_affine=True)
  (lm_head): RobertaLMHead(
    (dense): Linear(in_features=320, out_features=320, bias=True)
    (layer_norm): LayerNorm((320,), eps=1e-05, elementwise_affine=True)
  )
)
```

This cell takes quite a while to run locally; therefore writing the results to file for future use.

```
In [34]: # CELL_TAKE_TIME
# Comment out the following code if you have already created the embeddings_small.pkl file and don't want to re-run
# Create a dictionary of embeddings for all proteins, and write to file
# embedding_dict_small = {}

# # Loop through each protein in the set
# for gene in new_gene_list:
#     sequence = sequence_dictionary[gene]
#     embedding = get_esm_embedding(model_small, sequence, batch_converter)
```

```
#     embedding_dict_small[gene] = embedding

# # Write the embeddings dictionary to file
# with open('../data/processed/embeddings_small.pkl', 'wb') as file:
#     pickle.dump(embedding_dict_small, file)
```

```
In [35]: # Read the embedding dictionary from the binary file
with open('../data/processed/embeddings_small.pkl', 'rb') as file:
    embedding_dict_small = pickle.load(file)
```

Section 3 - Train the Model

In this section we train a basic feed-forward multi-layer perceptron (MLP).

```
In [36]: # First we need to create a test/train split
train_proteins, test_proteins = train_test_split(
    new_gene_list,
    test_size=0.20, #Train on 80% of the data, test on the other 20%
    random_state=42
)

# Now form test and train interaction matrices
MIM_train = MIM.loc[train_proteins, train_proteins]
MIM_test = MIM.loc[test_proteins, test_proteins]
```

```
In [37]: # Create test and train dataframes
X_train, y_train, train_pairs = create_balanced_dataset(
    MIM_train,
    embedding_dict,
    n=100000, # Number of positive and negative interactions to pull from the training interaction matrix
    random_state=42
)

X_test, y_test, train_pairs = create_balanced_dataset(
    MIM_test,
    embedding_dict,
    n=10000, # Number of positive and negative interactions to pull from the test interaction matrix
```

```
    random_state=42
)

# Scale the train and test sets
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [38]: # Define a simple neural network to start with
model = keras.Sequential([
    layers.Input(shape=(X_train.shape[1],)),

    layers.Dense(512, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(256, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(64, activation='relu'),

    layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=[
        'accuracy',
        keras.metrics.AUC(name='auc'),
        keras.metrics.Precision(name='precision'),
        keras.metrics.Recall(name='recall')
    ]
)

# Inspect the results
print(model.summary())
```

```

2026-08-19 01:58:35.714388: I metal_plugin/src/device/metal_device.cc:1154] Metal device set to: Apple M4
2026-08-19 01:58:35.715610: I metal_plugin/src/device/metal_device.cc:296] systemMemory: 16.00 GB
2026-08-19 01:58:35.715636: I metal_plugin/src/device/metal_device.cc:313] maxCacheSize: 5.92 GB
2026-08-19 01:58:35.716672: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:305] Could
not identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel may not have been built with NUMA support.
2026-08-19 01:58:35.716688: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271] Creat
ed TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0 MB memory) -> physical PluggableDevice (de
vice: 0, name: METAL, pci bus id: <undefined>)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	1,311,232
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 64)	16,448
dense_3 (Dense)	(None, 1)	65

Total params: 1,459,073 (5.57 MB)

Trainable params: 1,459,073 (5.57 MB)

Non-trainable params: 0 (0.00 B)

None

```

In [33]: # Train the simple neural network
history = model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    verbose=1
)

```

Epoch 1/100

```
2026-08-18 20:05:03.080040: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117] Plugin optimizer for device_type GPU is enabled.
```

5000/5000 ————— **79s** 15ms/step – accuracy: 0.5654 – auc: 0.5710 – loss: 4451.5854 – precision: 0.5601 – recall: 0.6119 – val_accuracy: 0.5993 – val_auc: 0.5983 – val_loss: 27158.2617 – val_precision: 0.5757 – val_recall: 0.7734
Epoch 2/100
5000/5000 ————— **78s** 16ms/step – accuracy: 0.5670 – auc: 0.5672 – loss: 87124.0391 – precision: 0.5584 – recall: 0.6311 – val_accuracy: 0.4961 – val_auc: 0.4968 – val_loss: 159136.3906 – val_precision: 0.4989 – val_recall: 0.3960
Epoch 3/100
5000/5000 ————— **74s** 15ms/step – accuracy: 0.5675 – auc: 0.5676 – loss: 281830.3750 – precision: 0.5595 – recall: 0.6306 – val_accuracy: 0.5750 – val_auc: 0.5740 – val_loss: 1218336.8750 – val_precision: 0.5584 – val_recall: 0.7420
Epoch 4/100
5000/5000 ————— **78s** 16ms/step – accuracy: 0.5655 – auc: 0.5656 – loss: 769018.6875 – precision: 0.5570 – recall: 0.6297 – val_accuracy: 0.6593 – val_auc: 0.6587 – val_loss: 210384.9531 – val_precision: 0.6343 – val_recall: 0.7620
Epoch 5/100
5000/5000 ————— **76s** 15ms/step – accuracy: 0.5686 – auc: 0.5688 – loss: 1386213.5000 – precision: 0.5592 – recall: 0.6352 – val_accuracy: 0.5777 – val_auc: 0.5766 – val_loss: 3815017.2500 – val_precision: 0.5602 – val_recall: 0.7459
Epoch 6/100
5000/5000 ————— **72s** 14ms/step – accuracy: 0.5694 – auc: 0.5693 – loss: 2554134.7500 – precision: 0.5621 – recall: 0.6318 – val_accuracy: 0.5834 – val_auc: 0.5824 – val_loss: 3991081.7500 – val_precision: 0.5643 – val_recall: 0.7541
Epoch 7/100
5000/5000 ————— **75s** 15ms/step – accuracy: 0.5674 – auc: 0.5675 – loss: 4233307.0000 – precision: 0.5594 – recall: 0.6269 – val_accuracy: 0.5722 – val_auc: 0.5712 – val_loss: 17995110.0000 – val_precision: 0.5563 – val_recall: 0.7385
Epoch 8/100
5000/5000 ————— **73s** 15ms/step – accuracy: 0.5668 – auc: 0.5672 – loss: 6196795.0000 – precision: 0.5570 – recall: 0.6283 – val_accuracy: 0.5946 – val_auc: 0.5935 – val_loss: 5347270.0000 – val_precision: 0.5721 – val_recall: 0.7691
Epoch 9/100
5000/5000 ————— **72s** 14ms/step – accuracy: 0.5673 – auc: 0.5671 – loss: 8541091.0000 – precision: 0.5607 – recall: 0.6352 – val_accuracy: 0.4793 – val_auc: 0.4800 – val_loss: 8952693.0000 – val_precision: 0.4765 – val_recall: 0.3584
Epoch 10/100
5000/5000 ————— **71s** 14ms/step – accuracy: 0.5706 – auc: 0.5708 – loss: 10391233.0000 – precision: 0.5618 – recall: 0.6314 – val_accuracy: 0.5792 – val_auc: 0.5781 – val_loss: 21916134.0000 – val_precision: 0.5612 – val_recall: 0.7488
Epoch 11/100
5000/5000 ————— **72s** 14ms/step – accuracy: 0.5655 – auc: 0.5657 – loss: 16082850.0000 – precision: 0.5

566 – recall: 0.6310 – val_accuracy: 0.6604 – val_auc: 0.6598 – val_loss: 3212651.7500 – val_precision: 0.6348 – val_recall: 0.7652
Epoch 12/100
5000/5000 ————— **72s** 14ms/step – accuracy: 0.5712 – auc: 0.5711 – loss: 20328918.0000 – precision: 0.5634 – recall: 0.6347 – val_accuracy: 0.4549 – val_auc: 0.4558 – val_loss: 41936156.0000 – val_precision: 0.4406 – val_recall: 0.3104
Epoch 13/100
5000/5000 ————— **74s** 15ms/step – accuracy: 0.5679 – auc: 0.5681 – loss: 26142674.0000 – precision: 0.5587 – recall: 0.6305 – val_accuracy: 0.5050 – val_auc: 0.5055 – val_loss: 15513915.0000 – val_precision: 0.5097 – val_recall: 0.4145
Epoch 14/100
5000/5000 ————— **85s** 17ms/step – accuracy: 0.5644 – auc: 0.5646 – loss: 33187594.0000 – precision: 0.5563 – recall: 0.6214 – val_accuracy: 0.5725 – val_auc: 0.5727 – val_loss: 7862702.0000 – val_precision: 0.5801 – val_recall: 0.5436
Epoch 15/100
5000/5000 ————— **79s** 16ms/step – accuracy: 0.5688 – auc: 0.5689 – loss: 40127528.0000 – precision: 0.5605 – recall: 0.6295 – val_accuracy: 0.5975 – val_auc: 0.5964 – val_loss: 28070544.0000 – val_precision: 0.5744 – val_recall: 0.7710
Epoch 16/100
5000/5000 ————— **75s** 15ms/step – accuracy: 0.5679 – auc: 0.5679 – loss: 51951972.0000 – precision: 0.5601 – recall: 0.6311 – val_accuracy: 0.6125 – val_auc: 0.6115 – val_loss: 22958454.0000 – val_precision: 0.5861 – val_recall: 0.7819
Epoch 17/100
5000/5000 ————— **75s** 15ms/step – accuracy: 0.5651 – auc: 0.5654 – loss: 64863968.0000 – precision: 0.5556 – recall: 0.6267 – val_accuracy: 0.5797 – val_auc: 0.5786 – val_loss: 105571664.0000 – val_precision: 0.5615 – val_recall: 0.7500
Epoch 18/100
5000/5000 ————— **79s** 16ms/step – accuracy: 0.5666 – auc: 0.5665 – loss: 68836728.0000 – precision: 0.5591 – recall: 0.6318 – val_accuracy: 0.4886 – val_auc: 0.4893 – val_loss: 57878120.0000 – val_precision: 0.4893 – val_recall: 0.3786
Epoch 19/100
5000/5000 ————— **80s** 16ms/step – accuracy: 0.5703 – auc: 0.5704 – loss: 73196432.0000 – precision: 0.5612 – recall: 0.6381 – val_accuracy: 0.4505 – val_auc: 0.4513 – val_loss: 197891536.0000 – val_precision: 0.4336 – val_recall: 0.3024
Epoch 20/100
5000/5000 ————— **88s** 18ms/step – accuracy: 0.5677 – auc: 0.5677 – loss: 96946264.0000 – precision: 0.5603 – recall: 0.6290 – val_accuracy: 0.5815 – val_auc: 0.5805 – val_loss: 139864480.0000 – val_precision: 0.5629 – val_recall: 0.7523
Epoch 21/100
5000/5000 ————— **77s** 15ms/step – accuracy: 0.5686 – auc: 0.5688 – loss: 113519560.0000 – precision: 0.5592 – recall: 0.6318 – val_accuracy: 0.6173 – val_auc: 0.6175 – val_loss: 18401088.0000 – val_precision: 0.6266 – val_recall: 0.7523

al_recall: 0.5919
Epoch 22/100
5000/5000  **85s** 17ms/step - accuracy: 0.5677 - auc: 0.5677 - loss: 139721584.0000 - precision: 0.5599 - recall: 0.6332 - val_accuracy: 0.6674 - val_auc: 0.6669 - val_loss: 19714786.0000 - val_precision: 0.6484 - val_recall: 0.7398
Epoch 23/100
5000/5000  **84s** 17ms/step - accuracy: 0.5679 - auc: 0.5680 - loss: 138377120.0000 - precision: 0.5598 - recall: 0.6297 - val_accuracy: 0.5879 - val_auc: 0.5869 - val_loss: 143653056.0000 - val_precision: 0.5675 - val_recall: 0.7600
Epoch 24/100
5000/5000  **86s** 17ms/step - accuracy: 0.5679 - auc: 0.5680 - loss: 168866656.0000 - precision: 0.5593 - recall: 0.6314 - val_accuracy: 0.6576 - val_auc: 0.6576 - val_loss: 22733770.0000 - val_precision: 0.6587 - val_recall: 0.6629
Epoch 25/100
5000/5000  **84s** 17ms/step - accuracy: 0.5660 - auc: 0.5660 - loss: 205222688.0000 - precision: 0.5586 - recall: 0.6321 - val_accuracy: 0.4701 - val_auc: 0.4708 - val_loss: 231764784.0000 - val_precision: 0.4634 - val_recall: 0.3388
Epoch 26/100
5000/5000  **82s** 16ms/step - accuracy: 0.5671 - auc: 0.5674 - loss: 240106368.0000 - precision: 0.5580 - recall: 0.6302 - val_accuracy: 0.6378 - val_auc: 0.6369 - val_loss: 59390972.0000 - val_precision: 0.6092 - val_recall: 0.7806
Epoch 27/100
5000/5000  **81s** 16ms/step - accuracy: 0.5691 - auc: 0.5694 - loss: 232862640.0000 - precision: 0.5599 - recall: 0.6315 - val_accuracy: 0.5408 - val_auc: 0.5411 - val_loss: 84704344.0000 - val_precision: 0.5492 - val_recall: 0.4863
Epoch 28/100
5000/5000  **77s** 15ms/step - accuracy: 0.5730 - auc: 0.5730 - loss: 244135600.0000 - precision: 0.5652 - recall: 0.6315 - val_accuracy: 0.5968 - val_auc: 0.5958 - val_loss: 190747504.0000 - val_precision: 0.5744 - val_recall: 0.7664
Epoch 29/100
5000/5000  **79s** 16ms/step - accuracy: 0.5713 - auc: 0.5714 - loss: 315348096.0000 - precision: 0.5627 - recall: 0.6330 - val_accuracy: 0.6655 - val_auc: 0.6649 - val_loss: 50574912.0000 - val_precision: 0.6429 - val_recall: 0.7535
Epoch 30/100
5000/5000  **79s** 16ms/step - accuracy: 0.5690 - auc: 0.5692 - loss: 308128928.0000 - precision: 0.5599 - recall: 0.6284 - val_accuracy: 0.5773 - val_auc: 0.5762 - val_loss: 728028864.0000 - val_precision: 0.5600 - val_recall: 0.7448
Epoch 31/100
5000/5000  **76s** 15ms/step - accuracy: 0.5714 - auc: 0.5713 - loss: 378132288.0000 - precision: 0.5649 - recall: 0.6300 - val_accuracy: 0.6201 - val_auc: 0.6203 - val_loss: 61717288.0000 - val_precision: 0.6309 - val_recall: 0.5897

Epoch 32/100

5000/5000  **82s** 16ms/step - accuracy: 0.5713 - auc: 0.5713 - loss: 380079264.0000 - precision: 0.5638 - recall: 0.6284 - val_accuracy: 0.5239 - val_auc: 0.5244 - val_loss: 176801456.0000 - val_precision: 0.5319 - val_recall: 0.4472

Epoch 33/100

5000/5000  **76s** 15ms/step - accuracy: 0.5712 - auc: 0.5715 - loss: 452745664.0000 - precision: 0.5611 - recall: 0.6300 - val_accuracy: 0.5881 - val_auc: 0.5871 - val_loss: 473106912.0000 - val_precision: 0.5679 - val_recall: 0.7576

Epoch 34/100

5000/5000  **77s** 15ms/step - accuracy: 0.5723 - auc: 0.5724 - loss: 436366240.0000 - precision: 0.5642 - recall: 0.6320 - val_accuracy: 0.5808 - val_auc: 0.5798 - val_loss: 755588352.0000 - val_precision: 0.5626 - val_recall: 0.7489

Epoch 35/100

5000/5000  **78s** 16ms/step - accuracy: 0.5736 - auc: 0.5737 - loss: 532181632.0000 - precision: 0.5653 - recall: 0.6292 - val_accuracy: 0.5228 - val_auc: 0.5232 - val_loss: 246759008.0000 - val_precision: 0.5306 - val_recall: 0.4436

Epoch 36/100

5000/5000  **76s** 15ms/step - accuracy: 0.5754 - auc: 0.5755 - loss: 519651904.0000 - precision: 0.5664 - recall: 0.6356 - val_accuracy: 0.4706 - val_auc: 0.4714 - val_loss: 664587008.0000 - val_precision: 0.4637 - val_recall: 0.3353

Epoch 37/100

5000/5000  **71s** 14ms/step - accuracy: 0.5706 - auc: 0.5705 - loss: 686263232.0000 - precision: 0.5641 - recall: 0.6276 - val_accuracy: 0.6060 - val_auc: 0.6050 - val_loss: 342457760.0000 - val_precision: 0.5817 - val_recall: 0.7714

Epoch 38/100

5000/5000  **71s** 14ms/step - accuracy: 0.5739 - auc: 0.5738 - loss: 654029632.0000 - precision: 0.5669 - recall: 0.6298 - val_accuracy: 0.6218 - val_auc: 0.6209 - val_loss: 258371280.0000 - val_precision: 0.5949 - val_recall: 0.7779

Epoch 39/100

5000/5000  **71s** 14ms/step - accuracy: 0.5711 - auc: 0.5715 - loss: 845273024.0000 - precision: 0.5609 - recall: 0.6318 - val_accuracy: 0.6110 - val_auc: 0.6100 - val_loss: 359908448.0000 - val_precision: 0.5859 - val_recall: 0.7732

Epoch 40/100

5000/5000  **70s** 14ms/step - accuracy: 0.5728 - auc: 0.5727 - loss: 797157568.0000 - precision: 0.5655 - recall: 0.6325 - val_accuracy: 0.5247 - val_auc: 0.5252 - val_loss: 355859648.0000 - val_precision: 0.5329 - val_recall: 0.4455

Epoch 41/100

5000/5000  **69s** 14ms/step - accuracy: 0.5719 - auc: 0.5720 - loss: 803318208.0000 - precision: 0.5641 - recall: 0.6291 - val_accuracy: 0.5914 - val_auc: 0.5904 - val_loss: 720125952.0000 - val_precision: 0.5705 - val_recall: 0.7595

Epoch 42/100

5000/5000 ————— **71s** 14ms/step – accuracy: 0.5688 – auc: 0.5690 – loss: 1076044160.0000 – precision: 0.5602 – recall: 0.6254 – val_accuracy: 0.5739 – val_auc: 0.5730 – val_loss: 3014129152.0000 – val_precision: 0.5577 – val_recall: 0.7396
Epoch 43/100

5000/5000 ————— **71s** 14ms/step – accuracy: 0.5696 – auc: 0.5695 – loss: 970101056.0000 – precision: 0.5629 – recall: 0.6262 – val_accuracy: 0.4612 – val_auc: 0.4621 – val_loss: 1457616896.0000 – val_precision: 0.4495 – val_recall: 0.3170
Epoch 44/100

5000/5000 ————— **73s** 15ms/step – accuracy: 0.5685 – auc: 0.5686 – loss: 1111565696.0000 – precision: 0.5605 – recall: 0.6229 – val_accuracy: 0.4521 – val_auc: 0.4530 – val_loss: 2344643328.0000 – val_precision: 0.4356 – val_recall: 0.3021
Epoch 45/100

5000/5000 ————— **70s** 14ms/step – accuracy: 0.5743 – auc: 0.5746 – loss: 1051614848.0000 – precision: 0.5640 – recall: 0.6324 – val_accuracy: 0.5943 – val_auc: 0.5932 – val_loss: 918706304.0000 – val_precision: 0.5726 – val_recall: 0.7623
Epoch 46/100

5000/5000 ————— **74s** 15ms/step – accuracy: 0.5715 – auc: 0.5716 – loss: 1132965248.0000 – precision: 0.5628 – recall: 0.6316 – val_accuracy: 0.4475 – val_auc: 0.4484 – val_loss: 3369754880.0000 – val_precision: 0.4283 – val_recall: 0.2940
Epoch 47/100

5000/5000 ————— **74s** 15ms/step – accuracy: 0.5743 – auc: 0.5746 – loss: 1232485888.0000 – precision: 0.5648 – recall: 0.6335 – val_accuracy: 0.5919 – val_auc: 0.5909 – val_loss: 1078536064.0000 – val_precision: 0.5709 – val_recall: 0.7594
Epoch 48/100

5000/5000 ————— **78s** 16ms/step – accuracy: 0.5696 – auc: 0.5695 – loss: 1444992640.0000 – precision: 0.5628 – recall: 0.6286 – val_accuracy: 0.5822 – val_auc: 0.5812 – val_loss: 1964171776.0000 – val_precision: 0.5637 – val_recall: 0.7497
Epoch 49/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5766 – auc: 0.5765 – loss: 1460611840.0000 – precision: 0.5690 – recall: 0.6352 – val_accuracy: 0.6453 – val_auc: 0.6454 – val_loss: 217285440.0000 – val_precision: 0.6551 – val_recall: 0.6227
Epoch 50/100

5000/5000 ————— **67s** 13ms/step – accuracy: 0.5689 – auc: 0.5691 – loss: 1688143360.0000 – precision: 0.5604 – recall: 0.6256 – val_accuracy: 0.6530 – val_auc: 0.6523 – val_loss: 328993984.0000 – val_precision: 0.6270 – val_recall: 0.7657
Epoch 51/100

5000/5000 ————— **66s** 13ms/step – accuracy: 0.5742 – auc: 0.5741 – loss: 1626193408.0000 – precision: 0.5668 – recall: 0.6271 – val_accuracy: 0.5881 – val_auc: 0.5871 – val_loss: 1679871104.0000 – val_precision: 0.5681 – val_recall: 0.7552
Epoch 52/100

5000/5000 ————— **66s** 13ms/step – accuracy: 0.5741 – auc: 0.5742 – loss: 1821598080.0000 – precision:

0.5655 - recall: 0.6277 - val_accuracy: 0.5764 - val_auc: 0.5754 - val_loss: 4304710144.0000 - val_precision: 0.5595
- val_recall: 0.7421
Epoch 53/100
5000/5000  **68s** 14ms/step - accuracy: 0.5691 - auc: 0.5693 - loss: 1851456128.0000 - precision:
0.5610 - recall: 0.6265 - val_accuracy: 0.4706 - val_auc: 0.4715 - val_loss: 2080135040.0000 - val_precision: 0.4636
- val_recall: 0.3337
Epoch 54/100
5000/5000  **77s** 15ms/step - accuracy: 0.5753 - auc: 0.5752 - loss: 1809025664.0000 - precision:
0.5685 - recall: 0.6293 - val_accuracy: 0.6183 - val_auc: 0.6174 - val_loss: 789239296.0000 - val_precision: 0.5921
- val_recall: 0.7752
Epoch 55/100
5000/5000  **75s** 15ms/step - accuracy: 0.5682 - auc: 0.5683 - loss: 2229652224.0000 - precision:
0.5606 - recall: 0.6247 - val_accuracy: 0.5990 - val_auc: 0.5981 - val_loss: 1371599232.0000 - val_precision: 0.5766
- val_recall: 0.7639
Epoch 56/100
5000/5000  **78s** 16ms/step - accuracy: 0.5754 - auc: 0.5755 - loss: 2202067712.0000 - precision:
0.5680 - recall: 0.6275 - val_accuracy: 0.6081 - val_auc: 0.6083 - val_loss: 391842432.0000 - val_precision: 0.6198
- val_recall: 0.5714
Epoch 57/100
5000/5000  **74s** 15ms/step - accuracy: 0.5723 - auc: 0.5723 - loss: 2241805824.0000 - precision:
0.5661 - recall: 0.6259 - val_accuracy: 0.4736 - val_auc: 0.4744 - val_loss: 2382096896.0000 - val_precision: 0.4680
- val_recall: 0.3397
Epoch 58/100
5000/5000  **73s** 15ms/step - accuracy: 0.5728 - auc: 0.5729 - loss: 2321156352.0000 - precision:
0.5650 - recall: 0.6253 - val_accuracy: 0.5118 - val_auc: 0.5124 - val_loss: 1250903680.0000 - val_precision: 0.5184
- val_recall: 0.4153
Epoch 59/100
5000/5000  **74s** 15ms/step - accuracy: 0.5701 - auc: 0.5703 - loss: 2627697920.0000 - precision:
0.5624 - recall: 0.6257 - val_accuracy: 0.5783 - val_auc: 0.5773 - val_loss: 5084957696.0000 - val_precision: 0.5609
- val_recall: 0.7440
Epoch 60/100
5000/5000  **71s** 14ms/step - accuracy: 0.5697 - auc: 0.5695 - loss: 2908789760.0000 - precision:
0.5638 - recall: 0.6233 - val_accuracy: 0.4615 - val_auc: 0.4624 - val_loss: 3853710592.0000 - val_precision: 0.4499
- val_recall: 0.3167
Epoch 61/100
5000/5000  **71s** 14ms/step - accuracy: 0.5697 - auc: 0.5695 - loss: 2908168960.0000 - precision:
0.5642 - recall: 0.6247 - val_accuracy: 0.6736 - val_auc: 0.6734 - val_loss: 382114528.0000 - val_precision: 0.6694
- val_recall: 0.6934
Epoch 62/100
5000/5000  **76s** 15ms/step - accuracy: 0.5765 - auc: 0.5766 - loss: 2618202368.0000 - precision:
0.5686 - recall: 0.6299 - val_accuracy: 0.5778 - val_auc: 0.5768 - val_loss: 6149961728.0000 - val_precision: 0.5606

- val_recall: 0.7431
Epoch 63/100
5000/5000  **74s** 15ms/step - accuracy: 0.5744 - auc: 0.5746 - loss: 3090650112.0000 - precision: 0.5650 - recall: 0.6319 - val_accuracy: 0.6099 - val_auc: 0.6090 - val_loss: 1525477120.0000 - val_precision: 0.5853
- val_recall: 0.7699
Epoch 64/100
5000/5000  **72s** 14ms/step - accuracy: 0.5725 - auc: 0.5726 - loss: 3049974784.0000 - precision: 0.5638 - recall: 0.6271 - val_accuracy: 0.4410 - val_auc: 0.4419 - val_loss: 14319892480.0000 - val_precision: 0.4179 - val_recall: 0.2835
Epoch 65/100
5000/5000  **72s** 14ms/step - accuracy: 0.5714 - auc: 0.5717 - loss: 3388319488.0000 - precision: 0.5623 - recall: 0.6247 - val_accuracy: 0.6091 - val_auc: 0.6094 - val_loss: 637903552.0000 - val_precision: 0.6211 - val_recall: 0.5718
Epoch 66/100
5000/5000  **70s** 14ms/step - accuracy: 0.5741 - auc: 0.5741 - loss: 3446419456.0000 - precision: 0.5668 - recall: 0.6311 - val_accuracy: 0.5885 - val_auc: 0.5874 - val_loss: 3834437632.0000 - val_precision: 0.5684 - val_recall: 0.7559
Epoch 67/100
5000/5000  **71s** 14ms/step - accuracy: 0.5752 - auc: 0.5754 - loss: 3827146240.0000 - precision: 0.5661 - recall: 0.6299 - val_accuracy: 0.6107 - val_auc: 0.6097 - val_loss: 1835552640.0000 - val_precision: 0.5860 - val_recall: 0.7695
Epoch 68/100
5000/5000  **80s** 16ms/step - accuracy: 0.5716 - auc: 0.5717 - loss: 4192437504.0000 - precision: 0.5642 - recall: 0.6285 - val_accuracy: 0.5128 - val_auc: 0.5133 - val_loss: 2176215296.0000 - val_precision: 0.5195 - val_recall: 0.4172
Epoch 69/100
5000/5000  **74s** 15ms/step - accuracy: 0.5736 - auc: 0.5736 - loss: 3866408960.0000 - precision: 0.5662 - recall: 0.6287 - val_accuracy: 0.6050 - val_auc: 0.6040 - val_loss: 2317707264.0000 - val_precision: 0.5814 - val_recall: 0.7672
Epoch 70/100
5000/5000  **69s** 14ms/step - accuracy: 0.5741 - auc: 0.5743 - loss: 4144781056.0000 - precision: 0.5652 - recall: 0.6295 - val_accuracy: 0.4633 - val_auc: 0.4642 - val_loss: 6201719296.0000 - val_precision: 0.4527 - val_recall: 0.3207
Epoch 71/100
5000/5000  **71s** 14ms/step - accuracy: 0.5755 - auc: 0.5757 - loss: 4175844352.0000 - precision: 0.5665 - recall: 0.6324 - val_accuracy: 0.5055 - val_auc: 0.5061 - val_loss: 2630543616.0000 - val_precision: 0.5108 - val_recall: 0.4008
Epoch 72/100
5000/5000  **69s** 14ms/step - accuracy: 0.5759 - auc: 0.5759 - loss: 4529013760.0000 - precision: 0.5675 - recall: 0.6316 - val_accuracy: 0.6691 - val_auc: 0.6687 - val_loss: 758813696.0000 - val_precision: 0.6498 - val_recall: 0.7424

Epoch 73/100

5000/5000  **74s** 15ms/step – accuracy: 0.5762 – auc: 0.5765 – loss: 4491606016.0000 – precision: 0.5661 – recall: 0.6333 – val_accuracy: 0.5966 – val_auc: 0.5956 – val_loss: 3669638144.0000 – val_precision: 0.5747 – val_recall: 0.7619

Epoch 74/100

5000/5000  **72s** 14ms/step – accuracy: 0.5744 – auc: 0.5743 – loss: 5682108928.0000 – precision: 0.5679 – recall: 0.6325 – val_accuracy: 0.5335 – val_auc: 0.5339 – val_loss: 1999265536.0000 – val_precision: 0.5430 – val_recall: 0.4575

Epoch 75/100

5000/5000  **72s** 14ms/step – accuracy: 0.5749 – auc: 0.5748 – loss: 4962575872.0000 – precision: 0.5680 – recall: 0.6312 – val_accuracy: 0.5902 – val_auc: 0.5892 – val_loss: 4992185856.0000 – val_precision: 0.5698 – val_recall: 0.7561

Epoch 76/100

5000/5000  **72s** 14ms/step – accuracy: 0.5705 – auc: 0.5704 – loss: 5066235392.0000 – precision: 0.5651 – recall: 0.6241 – val_accuracy: 0.5217 – val_auc: 0.5223 – val_loss: 2490505472.0000 – val_precision: 0.5301 – val_recall: 0.4329

Epoch 77/100

5000/5000  **72s** 14ms/step – accuracy: 0.5736 – auc: 0.5734 – loss: 5404822016.0000 – precision: 0.5673 – recall: 0.6308 – val_accuracy: 0.4729 – val_auc: 0.4737 – val_loss: 6003461632.0000 – val_precision: 0.4668 – val_recall: 0.3367

Epoch 78/100

5000/5000  **72s** 14ms/step – accuracy: 0.5777 – auc: 0.5777 – loss: 5286933504.0000 – precision: 0.5707 – recall: 0.6282 – val_accuracy: 0.6162 – val_auc: 0.6165 – val_loss: 1016110080.0000 – val_precision: 0.6289 – val_recall: 0.5783

Epoch 79/100

5000/5000  **72s** 14ms/step – accuracy: 0.5689 – auc: 0.5689 – loss: 7093442560.0000 – precision: 0.5625 – recall: 0.6242 – val_accuracy: 0.5954 – val_auc: 0.5944 – val_loss: 4843260416.0000 – val_precision: 0.5739 – val_recall: 0.7598

Epoch 80/100

5000/5000  **72s** 14ms/step – accuracy: 0.5731 – auc: 0.5730 – loss: 6221019648.0000 – precision: 0.5659 – recall: 0.6289 – val_accuracy: 0.5779 – val_auc: 0.5769 – val_loss: 14237608960.0000 – val_precision: 0.5606 – val_recall: 0.7434

Epoch 81/100

5000/5000  **72s** 14ms/step – accuracy: 0.5721 – auc: 0.5720 – loss: 7196611072.0000 – precision: 0.5651 – recall: 0.6303 – val_accuracy: 0.5785 – val_auc: 0.5776 – val_loss: 13989822464.0000 – val_precision: 0.5611 – val_recall: 0.7442

Epoch 82/100

5000/5000  **72s** 14ms/step – accuracy: 0.5717 – auc: 0.5720 – loss: 7417924608.0000 – precision: 0.5626 – recall: 0.6272 – val_accuracy: 0.4570 – val_auc: 0.4579 – val_loss: 11372380160.0000 – val_precision: 0.4429 – val_recall: 0.3085

Epoch 83/100

5000/5000 ————— **73s** 15ms/step – accuracy: 0.5756 – auc: 0.5756 – loss: 6659150336.0000 – precision: 0.5684 – recall: 0.6300 – val_accuracy: 0.4740 – val_auc: 0.4748 – val_loss: 7142413312.0000 – val_precision: 0.4684 – val_recall: 0.3388
Epoch 84/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5708 – auc: 0.5708 – loss: 7589240320.0000 – precision: 0.5636 – recall: 0.6251 – val_accuracy: 0.4495 – val_auc: 0.4504 – val_loss: 17799409664.0000 – val_precision: 0.4314 – val_recall: 0.2969
Epoch 85/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5741 – auc: 0.5741 – loss: 7479332352.0000 – precision: 0.5660 – recall: 0.6316 – val_accuracy: 0.5245 – val_auc: 0.5250 – val_loss: 3413055744.0000 – val_precision: 0.5333 – val_recall: 0.4380
Epoch 86/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5730 – auc: 0.5731 – loss: 7851317248.0000 – precision: 0.5649 – recall: 0.6261 – val_accuracy: 0.6061 – val_auc: 0.6052 – val_loss: 4658397696.0000 – val_precision: 0.5824 – val_recall: 0.7667
Epoch 87/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5745 – auc: 0.5747 – loss: 8088980480.0000 – precision: 0.5653 – recall: 0.6302 – val_accuracy: 0.5785 – val_auc: 0.5776 – val_loss: 17751025664.0000 – val_precision: 0.5611 – val_recall: 0.7439
Epoch 88/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5787 – auc: 0.5786 – loss: 7801145856.0000 – precision: 0.5719 – recall: 0.6337 – val_accuracy: 0.6733 – val_auc: 0.6730 – val_loss: 1267929856.0000 – val_precision: 0.6575 – val_recall: 0.7320
Epoch 89/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5752 – auc: 0.5754 – loss: 8247912960.0000 – precision: 0.5657 – recall: 0.6312 – val_accuracy: 0.6048 – val_auc: 0.6039 – val_loss: 5322817536.0000 – val_precision: 0.5814 – val_recall: 0.7657
Epoch 90/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5737 – auc: 0.5739 – loss: 8976919552.0000 – precision: 0.5648 – recall: 0.6315 – val_accuracy: 0.5350 – val_auc: 0.5355 – val_loss: 3488737024.0000 – val_precision: 0.5451 – val_recall: 0.4569
Epoch 91/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5737 – auc: 0.5738 – loss: 9250059264.0000 – precision: 0.5664 – recall: 0.6270 – val_accuracy: 0.6052 – val_auc: 0.6043 – val_loss: 5548988928.0000 – val_precision: 0.5818 – val_recall: 0.7654
Epoch 92/100

5000/5000 ————— **73s** 15ms/step – accuracy: 0.5772 – auc: 0.5774 – loss: 8230761472.0000 – precision: 0.5677 – recall: 0.6318 – val_accuracy: 0.4766 – val_auc: 0.4774 – val_loss: 9818891264.0000 – val_precision: 0.4722 – val_recall: 0.3442
Epoch 93/100

5000/5000 ————— **72s** 14ms/step – accuracy: 0.5735 – auc: 0.5737 – loss: 9921801216.0000 – precision:

0.5649 - recall: 0.6267 - val_accuracy: 0.4699 - val_auc: 0.4708 - val_loss: 11633600512.0000 - val_precision: 0.4624 - val_recall: 0.3305
 Epoch 94/100
5000/5000  72s 14ms/step - accuracy: 0.5735 - auc: 0.5735 - loss: 9699884032.0000 - precision: 0.5664 - recall: 0.6262 - val_accuracy: 0.6010 - val_auc: 0.6000 - val_loss: 6743441408.0000 - val_precision: 0.5785 - val_recall: 0.7619
 Epoch 95/100
5000/5000  72s 14ms/step - accuracy: 0.5732 - auc: 0.5733 - loss: 10007912448.0000 - precision: 0.5650 - recall: 0.6278 - val_accuracy: 0.4696 - val_auc: 0.4704 - val_loss: 12668983296.0000 - val_precision: 0.4619 - val_recall: 0.3304
 Epoch 96/100
5000/5000  73s 15ms/step - accuracy: 0.5737 - auc: 0.5740 - loss: 11389299712.0000 - precision: 0.5644 - recall: 0.6287 - val_accuracy: 0.6126 - val_auc: 0.6116 - val_loss: 5195060736.0000 - val_precision: 0.5879 - val_recall: 0.7685
 Epoch 97/100
5000/5000  72s 14ms/step - accuracy: 0.5671 - auc: 0.5671 - loss: 13000334336.0000 - precision: 0.5619 - recall: 0.6191 - val_accuracy: 0.6471 - val_auc: 0.6464 - val_loss: 2794370560.0000 - val_precision: 0.6203 - val_recall: 0.7696
 Epoch 98/100
5000/5000  72s 14ms/step - accuracy: 0.5747 - auc: 0.5744 - loss: 11354808320.0000 - precision: 0.5695 - recall: 0.6265 - val_accuracy: 0.5878 - val_auc: 0.5868 - val_loss: 12505551872.0000 - val_precision: 0.5681 - val_recall: 0.7526
 Epoch 99/100
5000/5000  72s 14ms/step - accuracy: 0.5763 - auc: 0.5762 - loss: 10693220352.0000 - precision: 0.5689 - recall: 0.6331 - val_accuracy: 0.5109 - val_auc: 0.5115 - val_loss: 6565536256.0000 - val_precision: 0.5174 - val_recall: 0.4112
 Epoch 100/100
5000/5000  72s 14ms/step - accuracy: 0.5763 - auc: 0.5764 - loss: 11786263552.0000 - precision: 0.5684 - recall: 0.6272 - val_accuracy: 0.6103 - val_auc: 0.6094 - val_loss: 6193421312.0000 - val_precision: 0.5860 - val_recall: 0.7678

```
In [34]: # Save it for future use
model.save('../data/processed/ppi_esm2_model.keras')
```

Here we are training a second model with the same architecture as the original using the reduced sized embeddings.

```
In [35]: # Create test and train dataframes
X_train_small, y_train_small, train_pairs = create_balanced_dataset(
    MIM_train,
    embedding_dict_small,
```

```

    n=100000,      # Number of positive and negative interactions to pull from the training interaction matrix
    random_state=42
)

X_test_small, y_test_small, train_pairs = create_balanced_dataset(
    MIM_test,
    embedding_dict_small,
    n=10000,      # Number of positive and negative interactions to pull from the test interaction matrix
    random_state=42
)

# Scale the train and test sets
scaler = StandardScaler()

X_train_scaled_small = scaler.fit_transform(X_train_small)
X_test_scaled_small = scaler.transform(X_test_small)

```

```

In [36]: # debug
print(X_train_scaled_small.shape)
print(X_train_small.shape)
print(X_train_small[0].shape)

```

```

(200000, 640)
(200000, 640)
(640,)

```

```

In [37]: # Define a simple neural network to start with
model_small = keras.Sequential([
    layers.Input(shape=(X_train_small.shape[1],)),

    layers.Dense(256, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(32, activation='relu'),

    layers.Dense(1, activation='sigmoid')
])

```

```

model_small.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=[
        'accuracy',
        keras.metrics.AUC(name='auc'),
        keras.metrics.Precision(name='precision'),
        keras.metrics.Recall(name='recall')
    ]
)

# Inspect the results
print(model_small.summary())

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 256)	164,096
dropout_2 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 128)	32,896
dropout_3 (Dropout)	(None, 128)	0
dense_6 (Dense)	(None, 32)	4,128
dense_7 (Dense)	(None, 1)	33

Total params: 201,153 (785.75 KB)

Trainable params: 201,153 (785.75 KB)

Non-trainable params: 0 (0.00 B)

None

```

In [38]: # Train the simple neural network
history = model_small.fit(
    X_train_scaled_small,
    y_train_small,
    validation_split=0.2,

```

```
epochs=100,  
batch_size=32,  
verbose=1  
)
```

Epoch 1/100

5000/5000  **63s** 12ms/step - accuracy: 0.5617 - auc: 0.5767 - loss: 8.5486 - precision: 0.5586 - recall: 0.5729 - val_accuracy: 0.5568 - val_auc: 0.5558 - val_loss: 211.1068 - val_precision: 0.5444 - val_recall: 0.7288

Epoch 2/100

5000/5000  **60s** 12ms/step - accuracy: 0.5452 - auc: 0.5456 - loss: 522.1408 - precision: 0.5394 - recall: 0.5920 - val_accuracy: 0.5738 - val_auc: 0.5729 - val_loss: 770.9352 - val_precision: 0.5566 - val_recall: 0.7503

Epoch 3/100

5000/5000  **60s** 12ms/step - accuracy: 0.5480 - auc: 0.5481 - loss: 2980.3164 - precision: 0.5432 - recall: 0.6020 - val_accuracy: 0.5569 - val_auc: 0.5558 - val_loss: 6125.3779 - val_precision: 0.5441 - val_recall: 0.7340

Epoch 4/100

5000/5000  **60s** 12ms/step - accuracy: 0.5472 - auc: 0.5472 - loss: 8701.2520 - precision: 0.5426 - recall: 0.5999 - val_accuracy: 0.4777 - val_auc: 0.4787 - val_loss: 81052.1094 - val_precision: 0.4719 - val_recall: 0.3221

Epoch 5/100

5000/5000  **60s** 12ms/step - accuracy: 0.5503 - auc: 0.5503 - loss: 18046.1621 - precision: 0.5458 - recall: 0.6028 - val_accuracy: 0.5562 - val_auc: 0.5550 - val_loss: 22352.4648 - val_precision: 0.5434 - val_recall: 0.7367

Epoch 6/100

5000/5000  **60s** 12ms/step - accuracy: 0.5507 - auc: 0.5507 - loss: 34198.6250 - precision: 0.5455 - recall: 0.6021 - val_accuracy: 0.5447 - val_auc: 0.5437 - val_loss: 62063.8906 - val_precision: 0.5354 - val_recall: 0.7180

Epoch 7/100

5000/5000  **60s** 12ms/step - accuracy: 0.5477 - auc: 0.5480 - loss: 59709.5820 - precision: 0.5404 - recall: 0.5990 - val_accuracy: 0.6024 - val_auc: 0.6015 - val_loss: 24104.1055 - val_precision: 0.5799 - val_recall: 0.7609

Epoch 8/100

5000/5000  **60s** 12ms/step - accuracy: 0.5486 - auc: 0.5487 - loss: 102341.6406 - precision: 0.5435 - recall: 0.6018 - val_accuracy: 0.5444 - val_auc: 0.5434 - val_loss: 126950.5625 - val_precision: 0.5351 - val_recall: 0.7195

Epoch 9/100

5000/5000  **60s** 12ms/step - accuracy: 0.5460 - auc: 0.5460 - loss: 146301.8125 - precision: 0.5414 - recall: 0.5969 - val_accuracy: 0.5078 - val_auc: 0.5085 - val_loss: 138629.4062 - val_precision: 0.5145 - val_recall: 0.3813

Epoch 10/100

5000/5000  **60s** 12ms/step - accuracy: 0.5446 - auc: 0.5447 - loss: 197433.7656 - precision: 0.5398 - recall: 0.5950 - val_accuracy: 0.4911 - val_auc: 0.4920 - val_loss: 338051.0938 - val_precision: 0.4917 - val_recall: 0.3482

Epoch 11/100

5000/5000  **60s** 12ms/step – accuracy: 0.5480 – auc: 0.5480 – loss: 285556.3750 – precision: 0.5431 – recall: 0.5982 – val_accuracy: 0.5411 – val_auc: 0.5400 – val_loss: 493024.5625 – val_precision: 0.5327 – val_recall: 0.7141
Epoch 12/100

5000/5000  **60s** 12ms/step – accuracy: 0.5499 – auc: 0.5499 – loss: 347635.6562 – precision: 0.5453 – recall: 0.6016 – val_accuracy: 0.5415 – val_auc: 0.5404 – val_loss: 693126.2500 – val_precision: 0.5329 – val_recall: 0.7160
Epoch 13/100

5000/5000  **60s** 12ms/step – accuracy: 0.5453 – auc: 0.5453 – loss: 457815.1562 – precision: 0.5415 – recall: 0.5975 – val_accuracy: 0.5089 – val_auc: 0.5097 – val_loss: 357587.6562 – val_precision: 0.5159 – val_recall: 0.3836
Epoch 14/100

5000/5000  **60s** 12ms/step – accuracy: 0.5478 – auc: 0.5480 – loss: 701310.3750 – precision: 0.5416 – recall: 0.6022 – val_accuracy: 0.5339 – val_auc: 0.5329 – val_loss: 1479927.2500 – val_precision: 0.5276 – val_recall: 0.7016
Epoch 15/100

5000/5000  **60s** 12ms/step – accuracy: 0.5486 – auc: 0.5485 – loss: 824597.4375 – precision: 0.5449 – recall: 0.5995 – val_accuracy: 0.5202 – val_auc: 0.5209 – val_loss: 627739.3125 – val_precision: 0.5298 – val_recall: 0.4098
Epoch 16/100

5000/5000  **60s** 12ms/step – accuracy: 0.5534 – auc: 0.5532 – loss: 916353.8125 – precision: 0.5500 – recall: 0.6047 – val_accuracy: 0.5594 – val_auc: 0.5584 – val_loss: 719402.8750 – val_precision: 0.5457 – val_recall: 0.7416
Epoch 17/100

5000/5000  **60s** 12ms/step – accuracy: 0.5510 – auc: 0.5510 – loss: 1227636.0000 – precision: 0.5460 – recall: 0.6059 – val_accuracy: 0.5339 – val_auc: 0.5329 – val_loss: 3286554.0000 – val_precision: 0.5276 – val_recall: 0.7025
Epoch 18/100

5000/5000  **61s** 12ms/step – accuracy: 0.5506 – auc: 0.5505 – loss: 1351509.1250 – precision: 0.5463 – recall: 0.6055 – val_accuracy: 0.5421 – val_auc: 0.5410 – val_loss: 1672110.1250 – val_precision: 0.5333 – val_recall: 0.7167
Epoch 19/100

5000/5000  **60s** 12ms/step – accuracy: 0.5475 – auc: 0.5475 – loss: 1693493.5000 – precision: 0.5433 – recall: 0.5977 – val_accuracy: 0.5304 – val_auc: 0.5293 – val_loss: 6624837.0000 – val_precision: 0.5250 – val_recall: 0.6963
Epoch 20/100

5000/5000  **60s** 12ms/step – accuracy: 0.5430 – auc: 0.5430 – loss: 2119665.2500 – precision: 0.5385 – recall: 0.5967 – val_accuracy: 0.4920 – val_auc: 0.4929 – val_loss: 3370358.7500 – val_precision: 0.4930 – val_recall: 0.3478
Epoch 21/100

5000/5000  **61s** 12ms/step – accuracy: 0.5501 – auc: 0.5503 – loss: 2192769.7500 – precision: 0.54

36 – recall: 0.6006 – val_accuracy: 0.5601 – val_auc: 0.5590 – val_loss: 1249933.1250 – val_precision: 0.5460 – val_recall: 0.7439
Epoch 22/100
5000/5000  **61s** 12ms/step – accuracy: 0.5505 – auc: 0.5502 – loss: 2502194.5000 – precision: 0.5468 – recall: 0.6072 – val_accuracy: 0.5594 – val_auc: 0.5582 – val_loss: 1502165.8750 – val_precision: 0.5455 – val_recall: 0.7430
Epoch 23/100
5000/5000  **61s** 12ms/step – accuracy: 0.5475 – auc: 0.5476 – loss: 3250093.0000 – precision: 0.5430 – recall: 0.5999 – val_accuracy: 0.4886 – val_auc: 0.4896 – val_loss: 7697202.5000 – val_precision: 0.4880 – val_recall: 0.3390
Epoch 24/100
5000/5000  **61s** 12ms/step – accuracy: 0.5486 – auc: 0.5490 – loss: 3397050.2500 – precision: 0.5404 – recall: 0.6030 – val_accuracy: 0.5501 – val_auc: 0.5491 – val_loss: 2251561.2500 – val_precision: 0.5389 – val_recall: 0.7312
Epoch 25/100
5000/5000  **61s** 12ms/step – accuracy: 0.5458 – auc: 0.5461 – loss: 3859918.5000 – precision: 0.5390 – recall: 0.6023 – val_accuracy: 0.4864 – val_auc: 0.4874 – val_loss: 9387783.0000 – val_precision: 0.4848 – val_recall: 0.3349
Epoch 26/100
5000/5000  **61s** 12ms/step – accuracy: 0.5463 – auc: 0.5464 – loss: 5420436.0000 – precision: 0.5410 – recall: 0.5993 – val_accuracy: 0.5280 – val_auc: 0.5270 – val_loss: 18990622.0000 – val_precision: 0.5233 – val_recall: 0.6935
Epoch 27/100
5000/5000  **61s** 12ms/step – accuracy: 0.5436 – auc: 0.5437 – loss: 5465978.5000 – precision: 0.5388 – recall: 0.6007 – val_accuracy: 0.5363 – val_auc: 0.5352 – val_loss: 8273598.0000 – val_precision: 0.5292 – val_recall: 0.7083
Epoch 28/100
5000/5000  **61s** 12ms/step – accuracy: 0.5481 – auc: 0.5481 – loss: 5313737.0000 – precision: 0.5435 – recall: 0.6015 – val_accuracy: 0.5563 – val_auc: 0.5552 – val_loss: 4183788.0000 – val_precision: 0.5433 – val_recall: 0.7402
Epoch 29/100
5000/5000  **61s** 12ms/step – accuracy: 0.5505 – auc: 0.5506 – loss: 6346089.0000 – precision: 0.5445 – recall: 0.6034 – val_accuracy: 0.5010 – val_auc: 0.5018 – val_loss: 7209232.0000 – val_precision: 0.5055 – val_recall: 0.3660
Epoch 30/100
5000/5000  **61s** 12ms/step – accuracy: 0.5497 – auc: 0.5498 – loss: 6204397.0000 – precision: 0.5435 – recall: 0.6021 – val_accuracy: 0.4958 – val_auc: 0.4967 – val_loss: 10066053.0000 – val_precision: 0.4984 – val_recall: 0.3552
Epoch 31/100
5000/5000  **61s** 12ms/step – accuracy: 0.5510 – auc: 0.5512 – loss: 7254412.5000 – precision: 0.5449 – recall: 0.6020 – val_accuracy: 0.6176 – val_auc: 0.6179 – val_loss: 1575807.2500 – val_precision: 0.6303 – val_

recall: 0.5800

Epoch 32/100

5000/5000  **61s** 12ms/step - accuracy: 0.5489 - auc: 0.5491 - loss: 8367269.5000 - precision: 0.5433 - recall: 0.5977 - val_accuracy: 0.6094 - val_auc: 0.6097 - val_loss: 1388639.5000 - val_precision: 0.6254 - val_recall: 0.5569

Epoch 33/100

5000/5000  **61s** 12ms/step - accuracy: 0.5472 - auc: 0.5473 - loss: 9314182.0000 - precision: 0.5424 - recall: 0.5989 - val_accuracy: 0.5530 - val_auc: 0.5519 - val_loss: 5396915.5000 - val_precision: 0.5410 - val_recall: 0.7349

Epoch 34/100

5000/5000  **61s** 12ms/step - accuracy: 0.5485 - auc: 0.5486 - loss: 9767554.0000 - precision: 0.5429 - recall: 0.6041 - val_accuracy: 0.5426 - val_auc: 0.5415 - val_loss: 9265716.0000 - val_precision: 0.5336 - val_recall: 0.7194

Epoch 35/100

5000/5000  **61s** 12ms/step - accuracy: 0.5461 - auc: 0.5464 - loss: 11117988.0000 - precision: 0.5393 - recall: 0.6004 - val_accuracy: 0.4924 - val_auc: 0.4933 - val_loss: 19185792.0000 - val_precision: 0.4936 - val_recall: 0.3480

Epoch 36/100

5000/5000  **61s** 12ms/step - accuracy: 0.5459 - auc: 0.5459 - loss: 11651072.0000 - precision: 0.5409 - recall: 0.6033 - val_accuracy: 0.5150 - val_auc: 0.5158 - val_loss: 8018705.5000 - val_precision: 0.5237 - val_recall: 0.3960

Epoch 37/100

5000/5000  **61s** 12ms/step - accuracy: 0.5475 - auc: 0.5479 - loss: 13777979.0000 - precision: 0.5398 - recall: 0.6045 - val_accuracy: 0.5006 - val_auc: 0.5014 - val_loss: 18597756.0000 - val_precision: 0.5050 - val_recall: 0.3645

Epoch 38/100

5000/5000  **61s** 12ms/step - accuracy: 0.5544 - auc: 0.5546 - loss: 12837980.0000 - precision: 0.5482 - recall: 0.6107 - val_accuracy: 0.5520 - val_auc: 0.5510 - val_loss: 10836749.0000 - val_precision: 0.5403 - val_recall: 0.7333

Epoch 39/100

5000/5000  **61s** 12ms/step - accuracy: 0.5477 - auc: 0.5478 - loss: 14082772.0000 - precision: 0.5423 - recall: 0.5996 - val_accuracy: 0.4963 - val_auc: 0.4971 - val_loss: 20704360.0000 - val_precision: 0.4990 - val_recall: 0.3554

Epoch 40/100

5000/5000  **61s** 12ms/step - accuracy: 0.5486 - auc: 0.5486 - loss: 16650773.0000 - precision: 0.5436 - recall: 0.6020 - val_accuracy: 0.5335 - val_auc: 0.5340 - val_loss: 7889507.0000 - val_precision: 0.5449 - val_recall: 0.4393

Epoch 41/100

5000/5000  **61s** 12ms/step - accuracy: 0.5469 - auc: 0.5470 - loss: 16189162.0000 - precision: 0.5409 - recall: 0.6018 - val_accuracy: 0.5870 - val_auc: 0.5859 - val_loss: 5682895.5000 - val_precision: 0.5664 - val_recall: 0.7636

Epoch 42/100

5000/5000  **61s** 12ms/step – accuracy: 0.5483 – auc: 0.5484 – loss: 17904426.0000 – precision: 0.5426 – recall: 0.6030 – val_accuracy: 0.4913 – val_auc: 0.4922 – val_loss: 33947408.0000 – val_precision: 0.4920 – val_recall: 0.3452

Epoch 43/100

5000/5000  **61s** 12ms/step – accuracy: 0.5477 – auc: 0.5476 – loss: 21659484.0000 – precision: 0.5435 – recall: 0.6028 – val_accuracy: 0.5668 – val_auc: 0.5672 – val_loss: 5506260.0000 – val_precision: 0.5793 – val_recall: 0.5069

Epoch 44/100

5000/5000  **61s** 12ms/step – accuracy: 0.5483 – auc: 0.5484 – loss: 20573296.0000 – precision: 0.5424 – recall: 0.6040 – val_accuracy: 0.6093 – val_auc: 0.6084 – val_loss: 5333403.5000 – val_precision: 0.5874 – val_recall: 0.7503

Epoch 45/100

5000/5000  **61s** 12ms/step – accuracy: 0.5521 – auc: 0.5523 – loss: 22028314.0000 – precision: 0.5458 – recall: 0.6071 – val_accuracy: 0.5523 – val_auc: 0.5527 – val_loss: 9648905.0000 – val_precision: 0.5644 – val_recall: 0.4811

Epoch 46/100

5000/5000  **61s** 12ms/step – accuracy: 0.5536 – auc: 0.5538 – loss: 23767744.0000 – precision: 0.5464 – recall: 0.6115 – val_accuracy: 0.5334 – val_auc: 0.5324 – val_loss: 57349996.0000 – val_precision: 0.5272 – val_recall: 0.7023

Epoch 47/100

5000/5000  **61s** 12ms/step – accuracy: 0.5468 – auc: 0.5470 – loss: 28388144.0000 – precision: 0.5407 – recall: 0.5980 – val_accuracy: 0.5541 – val_auc: 0.5530 – val_loss: 18429412.0000 – val_precision: 0.5418 – val_recall: 0.7356

Epoch 48/100

5000/5000  **62s** 12ms/step – accuracy: 0.5498 – auc: 0.5499 – loss: 27387010.0000 – precision: 0.5442 – recall: 0.5990 – val_accuracy: 0.5885 – val_auc: 0.5874 – val_loss: 7717719.0000 – val_precision: 0.5677 – val_recall: 0.7627

Epoch 49/100

5000/5000  **62s** 12ms/step – accuracy: 0.5462 – auc: 0.5463 – loss: 27535234.0000 – precision: 0.5403 – recall: 0.6019 – val_accuracy: 0.4909 – val_auc: 0.4918 – val_loss: 73062544.0000 – val_precision: 0.4914 – val_recall: 0.3447

Epoch 50/100

5000/5000  **62s** 12ms/step – accuracy: 0.5488 – auc: 0.5490 – loss: 29905692.0000 – precision: 0.5417 – recall: 0.6029 – val_accuracy: 0.5722 – val_auc: 0.5711 – val_loss: 14842993.0000 – val_precision: 0.5548 – val_recall: 0.7570

Epoch 51/100

5000/5000  **62s** 12ms/step – accuracy: 0.5487 – auc: 0.5486 – loss: 33716908.0000 – precision: 0.5447 – recall: 0.6032 – val_accuracy: 0.5135 – val_auc: 0.5142 – val_loss: 29626922.0000 – val_precision: 0.5217 – val_recall: 0.3933

Epoch 52/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5495 – auc: 0.5496 – loss: 33510940.0000 – precision: 0.5437 – recall: 0.6004 – val_accuracy: 0.5335 – val_auc: 0.5324 – val_loss: 79627672.0000 – val_precision: 0.5272 – val_recall: 0.7021
Epoch 53/100

5000/5000 ————— **61s** 12ms/step – accuracy: 0.5494 – auc: 0.5494 – loss: 37184068.0000 – precision: 0.5445 – recall: 0.6047 – val_accuracy: 0.5397 – val_auc: 0.5386 – val_loss: 53919988.0000 – val_precision: 0.5316 – val_recall: 0.7136
Epoch 54/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5500 – auc: 0.5500 – loss: 34935108.0000 – precision: 0.5449 – recall: 0.6038 – val_accuracy: 0.5333 – val_auc: 0.5323 – val_loss: 88858496.0000 – val_precision: 0.5271 – val_recall: 0.7015
Epoch 55/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5476 – auc: 0.5477 – loss: 42053200.0000 – precision: 0.5422 – recall: 0.5996 – val_accuracy: 0.5040 – val_auc: 0.5048 – val_loss: 43533076.0000 – val_precision: 0.5095 – val_recall: 0.3711
Epoch 56/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5516 – auc: 0.5514 – loss: 44875056.0000 – precision: 0.5481 – recall: 0.6041 – val_accuracy: 0.5234 – val_auc: 0.5240 – val_loss: 25494342.0000 – val_precision: 0.5336 – val_recall: 0.4167
Epoch 57/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5464 – auc: 0.5464 – loss: 51961756.0000 – precision: 0.5411 – recall: 0.6000 – val_accuracy: 0.6198 – val_auc: 0.6190 – val_loss: 8081307.0000 – val_precision: 0.5994 – val_recall: 0.7359
Epoch 58/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5488 – auc: 0.5489 – loss: 49572812.0000 – precision: 0.5435 – recall: 0.6049 – val_accuracy: 0.5076 – val_auc: 0.5084 – val_loss: 51199128.0000 – val_precision: 0.5142 – val_recall: 0.3811
Epoch 59/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5489 – auc: 0.5490 – loss: 55160824.0000 – precision: 0.5438 – recall: 0.6021 – val_accuracy: 0.5027 – val_auc: 0.5036 – val_loss: 59103784.0000 – val_precision: 0.5079 – val_recall: 0.3690
Epoch 60/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5522 – auc: 0.5521 – loss: 54528792.0000 – precision: 0.5470 – recall: 0.6101 – val_accuracy: 0.4897 – val_auc: 0.4906 – val_loss: 109737232.0000 – val_precision: 0.4896 – val_recall: 0.3423
Epoch 61/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5450 – auc: 0.5450 – loss: 63474276.0000 – precision: 0.5403 – recall: 0.5994 – val_accuracy: 0.5041 – val_auc: 0.5049 – val_loss: 56396932.0000 – val_precision: 0.5096 – val_recall: 0.3715
Epoch 62/100

5000/5000 ————— **62s** 12ms/step – accuracy: 0.5467 – auc: 0.5468 – loss: 61135896.0000 – precision: 0.5

408 - recall: 0.6003 - val_accuracy: 0.4893 - val_auc: 0.4902 - val_loss: 158191984.0000 - val_precision: 0.4891 - val_recall: 0.3406
Epoch 63/100
5000/5000  **62s** 12ms/step - accuracy: 0.5506 - auc: 0.5507 - loss: 62227104.0000 - precision: 0.5442 - recall: 0.6014 - val_accuracy: 0.5760 - val_auc: 0.5749 - val_loss: 25386586.0000 - val_precision: 0.5577 - val_recall: 0.7594
Epoch 64/100
5000/5000  **62s** 12ms/step - accuracy: 0.5463 - auc: 0.5464 - loss: 77422816.0000 - precision: 0.5403 - recall: 0.6009 - val_accuracy: 0.5345 - val_auc: 0.5350 - val_loss: 31776254.0000 - val_precision: 0.5459 - val_recall: 0.4430
Epoch 65/100
5000/5000  **62s** 12ms/step - accuracy: 0.5486 - auc: 0.5485 - loss: 75198664.0000 - precision: 0.5440 - recall: 0.6059 - val_accuracy: 0.5832 - val_auc: 0.5821 - val_loss: 24583940.0000 - val_precision: 0.5633 - val_recall: 0.7624
Epoch 66/100
5000/5000  **62s** 12ms/step - accuracy: 0.5511 - auc: 0.5514 - loss: 66392212.0000 - precision: 0.5430 - recall: 0.6081 - val_accuracy: 0.5297 - val_auc: 0.5303 - val_loss: 33812104.0000 - val_precision: 0.5406 - val_recall: 0.4324
Epoch 67/100
5000/5000  **62s** 12ms/step - accuracy: 0.5448 - auc: 0.5450 - loss: 77610904.0000 - precision: 0.5386 - recall: 0.6016 - val_accuracy: 0.5370 - val_auc: 0.5360 - val_loss: 109822808.0000 - val_precision: 0.5297 - val_recall: 0.7081
Epoch 68/100
5000/5000  **62s** 12ms/step - accuracy: 0.5455 - auc: 0.5460 - loss: 86993776.0000 - precision: 0.5376 - recall: 0.5998 - val_accuracy: 0.6184 - val_auc: 0.6177 - val_loss: 14266524.0000 - val_precision: 0.5977 - val_recall: 0.7384
Epoch 69/100
5000/5000  **62s** 12ms/step - accuracy: 0.5472 - auc: 0.5475 - loss: 90074928.0000 - precision: 0.5403 - recall: 0.6007 - val_accuracy: 0.5734 - val_auc: 0.5723 - val_loss: 36477440.0000 - val_precision: 0.5557 - val_recall: 0.7572
Epoch 70/100
5000/5000  **62s** 12ms/step - accuracy: 0.5501 - auc: 0.5501 - loss: 81011744.0000 - precision: 0.5452 - recall: 0.6059 - val_accuracy: 0.5686 - val_auc: 0.5675 - val_loss: 38293356.0000 - val_precision: 0.5522 - val_recall: 0.7526
Epoch 71/100
5000/5000  **62s** 12ms/step - accuracy: 0.5479 - auc: 0.5480 - loss: 105338456.0000 - precision: 0.5425 - recall: 0.6016 - val_accuracy: 0.5398 - val_auc: 0.5388 - val_loss: 129751280.0000 - val_precision: 0.5317 - val_recall: 0.7146
Epoch 72/100
5000/5000  **62s** 12ms/step - accuracy: 0.5509 - auc: 0.5510 - loss: 89287472.0000 - precision: 0.5444 - recall: 0.6074 - val_accuracy: 0.5117 - val_auc: 0.5125 - val_loss: 80132312.0000 - val_precision: 0.5195 - va

l_recall: 0.3898

Epoch 73/100

5000/5000  **62s** 12ms/step - accuracy: 0.5465 - auc: 0.5465 - loss: 102091912.0000 - precision: 0.5420 - recall: 0.6027 - val_accuracy: 0.5063 - val_auc: 0.5071 - val_loss: 94686296.0000 - val_precision: 0.5125 - val_recall: 0.3781

Epoch 74/100

5000/5000  **62s** 12ms/step - accuracy: 0.5507 - auc: 0.5508 - loss: 106481792.0000 - precision: 0.5453 - recall: 0.6029 - val_accuracy: 0.5341 - val_auc: 0.5331 - val_loss: 224168608.0000 - val_precision: 0.5277 - val_recall: 0.7037

Epoch 75/100

5000/5000  **62s** 12ms/step - accuracy: 0.5495 - auc: 0.5494 - loss: 104652864.0000 - precision: 0.5453 - recall: 0.6026 - val_accuracy: 0.6192 - val_auc: 0.6194 - val_loss: 20526944.0000 - val_precision: 0.6305 - val_recall: 0.5871

Epoch 76/100

5000/5000  **62s** 12ms/step - accuracy: 0.5489 - auc: 0.5489 - loss: 122321032.0000 - precision: 0.5444 - recall: 0.6041 - val_accuracy: 0.6149 - val_auc: 0.6151 - val_loss: 22441886.0000 - val_precision: 0.6282 - val_recall: 0.5740

Epoch 77/100

5000/5000  **62s** 12ms/step - accuracy: 0.5534 - auc: 0.5535 - loss: 118664664.0000 - precision: 0.5476 - recall: 0.6052 - val_accuracy: 0.4966 - val_auc: 0.4974 - val_loss: 174357904.0000 - val_precision: 0.4994 - val_recall: 0.3559

Epoch 78/100

5000/5000  **62s** 12ms/step - accuracy: 0.5479 - auc: 0.5480 - loss: 128721728.0000 - precision: 0.5429 - recall: 0.6023 - val_accuracy: 0.5316 - val_auc: 0.5306 - val_loss: 310526496.0000 - val_precision: 0.5259 - val_recall: 0.6986

Epoch 79/100

5000/5000  **63s** 13ms/step - accuracy: 0.5450 - auc: 0.5451 - loss: 146563648.0000 - precision: 0.5397 - recall: 0.5962 - val_accuracy: 0.5365 - val_auc: 0.5355 - val_loss: 207197600.0000 - val_precision: 0.5294 - val_recall: 0.7078

Epoch 80/100

5000/5000  **63s** 13ms/step - accuracy: 0.5451 - auc: 0.5452 - loss: 147994192.0000 - precision: 0.5394 - recall: 0.5994 - val_accuracy: 0.5548 - val_auc: 0.5538 - val_loss: 85379832.0000 - val_precision: 0.5423 - val_recall: 0.7365

Epoch 81/100

5000/5000  **62s** 12ms/step - accuracy: 0.5450 - auc: 0.5452 - loss: 142252224.0000 - precision: 0.5396 - recall: 0.6016 - val_accuracy: 0.5306 - val_auc: 0.5296 - val_loss: 438523104.0000 - val_precision: 0.5251 - val_recall: 0.6970

Epoch 82/100

5000/5000  **62s** 12ms/step - accuracy: 0.5431 - auc: 0.5432 - loss: 147108320.0000 - precision: 0.5378 - recall: 0.6016 - val_accuracy: 0.5440 - val_auc: 0.5429 - val_loss: 144774928.0000 - val_precision: 0.5346 - val_recall: 0.7208

Epoch 83/100

5000/5000  **63s** 13ms/step - accuracy: 0.5519 - auc: 0.5521 - loss: 145924176.0000 - precision: 0.5452 - recall: 0.6072 - val_accuracy: 0.5343 - val_auc: 0.5333 - val_loss: 284754720.0000 - val_precision: 0.5278 - val_recall: 0.7038

Epoch 84/100

5000/5000  **63s** 13ms/step - accuracy: 0.5490 - auc: 0.5492 - loss: 151551920.0000 - precision: 0.5433 - recall: 0.6043 - val_accuracy: 0.4959 - val_auc: 0.4968 - val_loss: 203081760.0000 - val_precision: 0.4985 - val_recall: 0.3547

Epoch 85/100

5000/5000  **63s** 13ms/step - accuracy: 0.5464 - auc: 0.5463 - loss: 163341904.0000 - precision: 0.5431 - recall: 0.6000 - val_accuracy: 0.4863 - val_auc: 0.4872 - val_loss: 525113792.0000 - val_precision: 0.4845 - val_recall: 0.3348

Epoch 86/100

5000/5000  **63s** 13ms/step - accuracy: 0.5482 - auc: 0.5482 - loss: 178084176.0000 - precision: 0.5435 - recall: 0.6030 - val_accuracy: 0.5535 - val_auc: 0.5524 - val_loss: 125218312.0000 - val_precision: 0.5414 - val_recall: 0.7343

Epoch 87/100

5000/5000  **63s** 13ms/step - accuracy: 0.5527 - auc: 0.5526 - loss: 170206448.0000 - precision: 0.5482 - recall: 0.6090 - val_accuracy: 0.4870 - val_auc: 0.4880 - val_loss: 440389824.0000 - val_precision: 0.4857 - val_recall: 0.3367

Epoch 88/100

5000/5000  **63s** 13ms/step - accuracy: 0.5465 - auc: 0.5466 - loss: 175598848.0000 - precision: 0.5417 - recall: 0.5978 - val_accuracy: 0.5426 - val_auc: 0.5415 - val_loss: 189372000.0000 - val_precision: 0.5337 - val_recall: 0.7183

Epoch 89/100

5000/5000  **63s** 13ms/step - accuracy: 0.5501 - auc: 0.5499 - loss: 201042016.0000 - precision: 0.5470 - recall: 0.6045 - val_accuracy: 0.5565 - val_auc: 0.5554 - val_loss: 115882288.0000 - val_precision: 0.5436 - val_recall: 0.7382

Epoch 90/100

5000/5000  **63s** 13ms/step - accuracy: 0.5501 - auc: 0.5504 - loss: 203197584.0000 - precision: 0.5429 - recall: 0.6065 - val_accuracy: 0.5321 - val_auc: 0.5311 - val_loss: 478191136.0000 - val_precision: 0.5263 - val_recall: 0.6994

Epoch 91/100

5000/5000  **63s** 13ms/step - accuracy: 0.5465 - auc: 0.5466 - loss: 208947072.0000 - precision: 0.5411 - recall: 0.5999 - val_accuracy: 0.6347 - val_auc: 0.6345 - val_loss: 32234806.0000 - val_precision: 0.6287 - val_recall: 0.6683

Epoch 92/100

5000/5000  **63s** 13ms/step - accuracy: 0.5494 - auc: 0.5494 - loss: 217547936.0000 - precision: 0.5445 - recall: 0.6025 - val_accuracy: 0.6058 - val_auc: 0.6061 - val_loss: 47620800.0000 - val_precision: 0.6218 - val_recall: 0.5521

Epoch 93/100

```

5000/5000 ————— 63s 13ms/step - accuracy: 0.5537 - auc: 0.5538 - loss: 188164528.0000 - precision: 0.
5471 - recall: 0.6104 - val_accuracy: 0.5395 - val_auc: 0.5385 - val_loss: 276006336.0000 - val_precision: 0.5315 -
val_recall: 0.7131
Epoch 94/100
5000/5000 ————— 63s 13ms/step - accuracy: 0.5505 - auc: 0.5509 - loss: 194721152.0000 - precision: 0.
5423 - recall: 0.6062 - val_accuracy: 0.5569 - val_auc: 0.5558 - val_loss: 137397184.0000 - val_precision: 0.5438 -
val_recall: 0.7388
Epoch 95/100
5000/5000 ————— 63s 13ms/step - accuracy: 0.5463 - auc: 0.5465 - loss: 239739936.0000 - precision: 0.
5403 - recall: 0.5991 - val_accuracy: 0.6053 - val_auc: 0.6056 - val_loss: 44023624.0000 - val_precision: 0.6211 - v
al_recall: 0.5517
Epoch 96/100
5000/5000 ————— 63s 13ms/step - accuracy: 0.5489 - auc: 0.5491 - loss: 232216064.0000 - precision: 0.
5425 - recall: 0.6041 - val_accuracy: 0.5853 - val_auc: 0.5843 - val_loss: 71992464.0000 - val_precision: 0.5651 - v
al_recall: 0.7625
Epoch 97/100
5000/5000 ————— 63s 13ms/step - accuracy: 0.5478 - auc: 0.5478 - loss: 258606976.0000 - precision: 0.
5429 - recall: 0.6008 - val_accuracy: 0.5555 - val_auc: 0.5544 - val_loss: 138536320.0000 - val_precision: 0.5428 -
val_recall: 0.7373
Epoch 98/100
5000/5000 ————— 63s 13ms/step - accuracy: 0.5500 - auc: 0.5501 - loss: 223217888.0000 - precision: 0.
5445 - recall: 0.6027 - val_accuracy: 0.5357 - val_auc: 0.5346 - val_loss: 468308672.0000 - val_precision: 0.5288 -
val_recall: 0.7059
Epoch 99/100
5000/5000 ————— 63s 13ms/step - accuracy: 0.5503 - auc: 0.5503 - loss: 260708320.0000 - precision: 0.
5451 - recall: 0.6045 - val_accuracy: 0.5171 - val_auc: 0.5178 - val_loss: 165374992.0000 - val_precision: 0.5262 -
val_recall: 0.4028
Epoch 100/100
5000/5000 ————— 63s 13ms/step - accuracy: 0.5498 - auc: 0.5500 - loss: 269723040.0000 - precision: 0.
5440 - recall: 0.6011 - val_accuracy: 0.5848 - val_auc: 0.5837 - val_loss: 90424360.0000 - val_precision: 0.5646 - v
al_recall: 0.7624

```

```

In [39]: # Save it for future use
         model_small.save('../data/processed/ppi_esm2_model_small.keras')

```

Section 4 - Evaluate the Models

In this section we are evaluating the models, creating standard evaluation metrics for binary classifiers, and comparing the results models trained using 2 different pre-trained embedding models.

```
In [40]: # Re-load the previously trained models (as opposed to re-training them each time the notebook runs)
model1 = keras.models.load_model('../data/processed/ppi_esm2_model_no_scaling.keras')
model2 = keras.models.load_model('../data/processed/ppi_esm2_model.keras')
model_small = keras.models.load_model('../data/processed/ppi_esm2_model_small.keras')
```

Evaluate the first model trained without scaling the embedding vectors

```
In [41]: # Evaluate the model (1280-dimensional embedding, no scaling)
results = model1.evaluate(
    X_test,
    y_test,
    verbose=0
)

for name, value in zip(model1.metrics_names, results):
    print(f"{name}: {value:.4f}")

results1 = model1.evaluate(X_test, y_test, verbose=0, return_dict=True)
```

loss: 250.9064
compile_metrics: 0.5002

```
In [42]: y_prob1 = model1.predict(X_test).ravel()

roc_auc1 = roc_auc_score(y_test, y_prob1)
APS1 = average_precision_score(y_test, y_prob1)

print('ROC-AUC:', roc_auc1)
print('PR-AUC:', APS1)
```

625/625  1s 1ms/step
ROC-AUC: 0.5006
PR-AUC: 0.5005999999999999

```
In [43]: y_pred_labels1 = (y_prob1 >= 0.5).astype(int)
```

```
f1_1 = f1_score(y_test, y_pred_labels1)

print('f1 Score:', f1_1)
```

f1 Score: 0.0007996801279488205

```
In [44]: # Calculate ROC curve (without scaling)
fpr, tpr, thresholds = roc_curve(y_test, y_prob1)

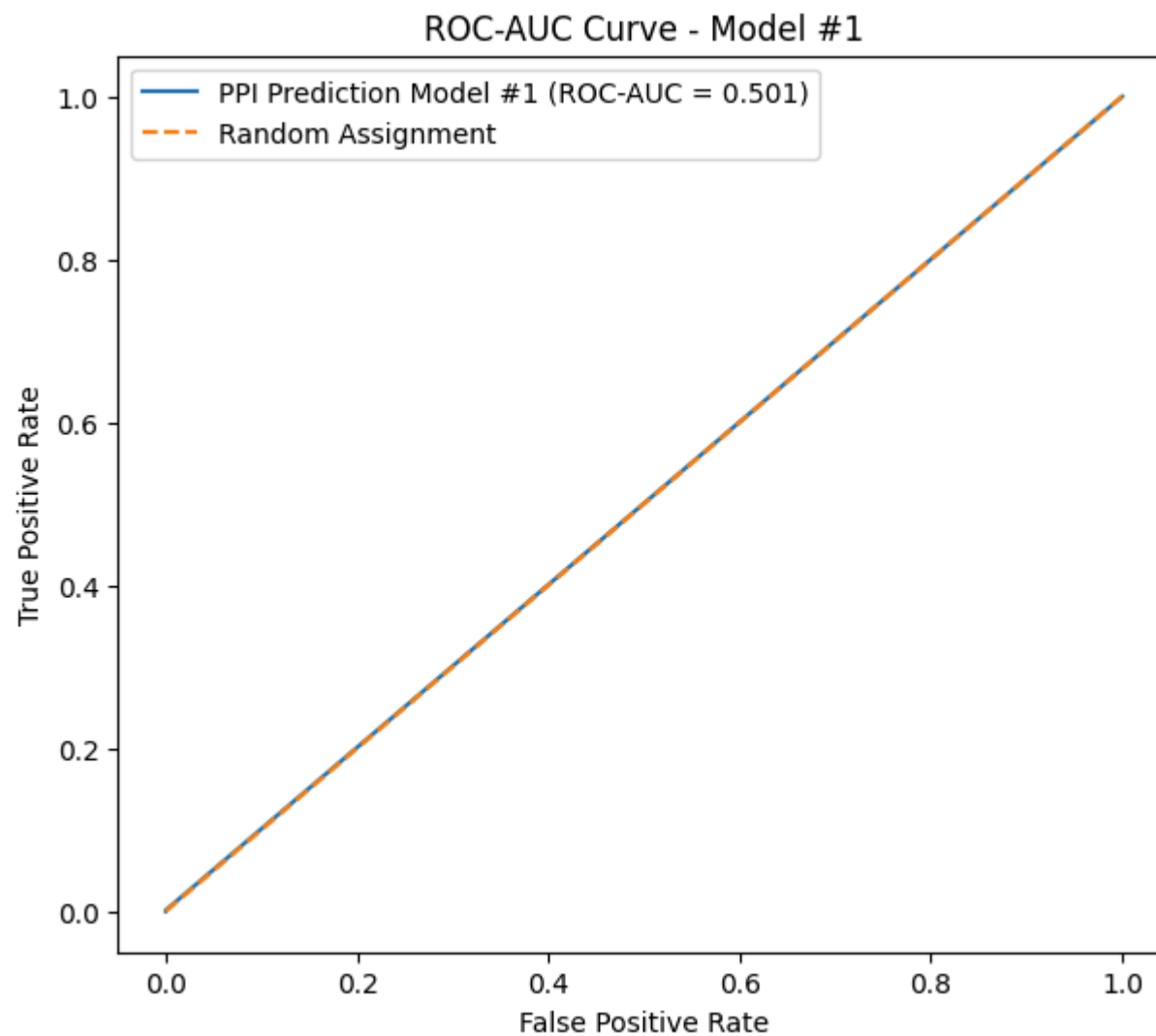
# Calculate AUROC
auc = roc_auc_score(y_test, y_prob1)

# Plot
plt.figure(figsize=(7, 6))

plt.plot(fpr, tpr, label=f'PPI Prediction Model #1 (ROC-AUC = {auc:.3f})')

# Random classifier
plt.plot([0, 1], [0, 1], linestyle='--', label='Random Assignment')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-AUC Curve - Model #1')
plt.legend()
plt.show()
```



```
In [45]: # Calculate precision-recall curve (without scaling)
precision, recall, thresholds = precision_recall_curve(y_test, y_prob1)

# Calculate AUPRC
auprc = average_precision_score(y_test, y_prob1)

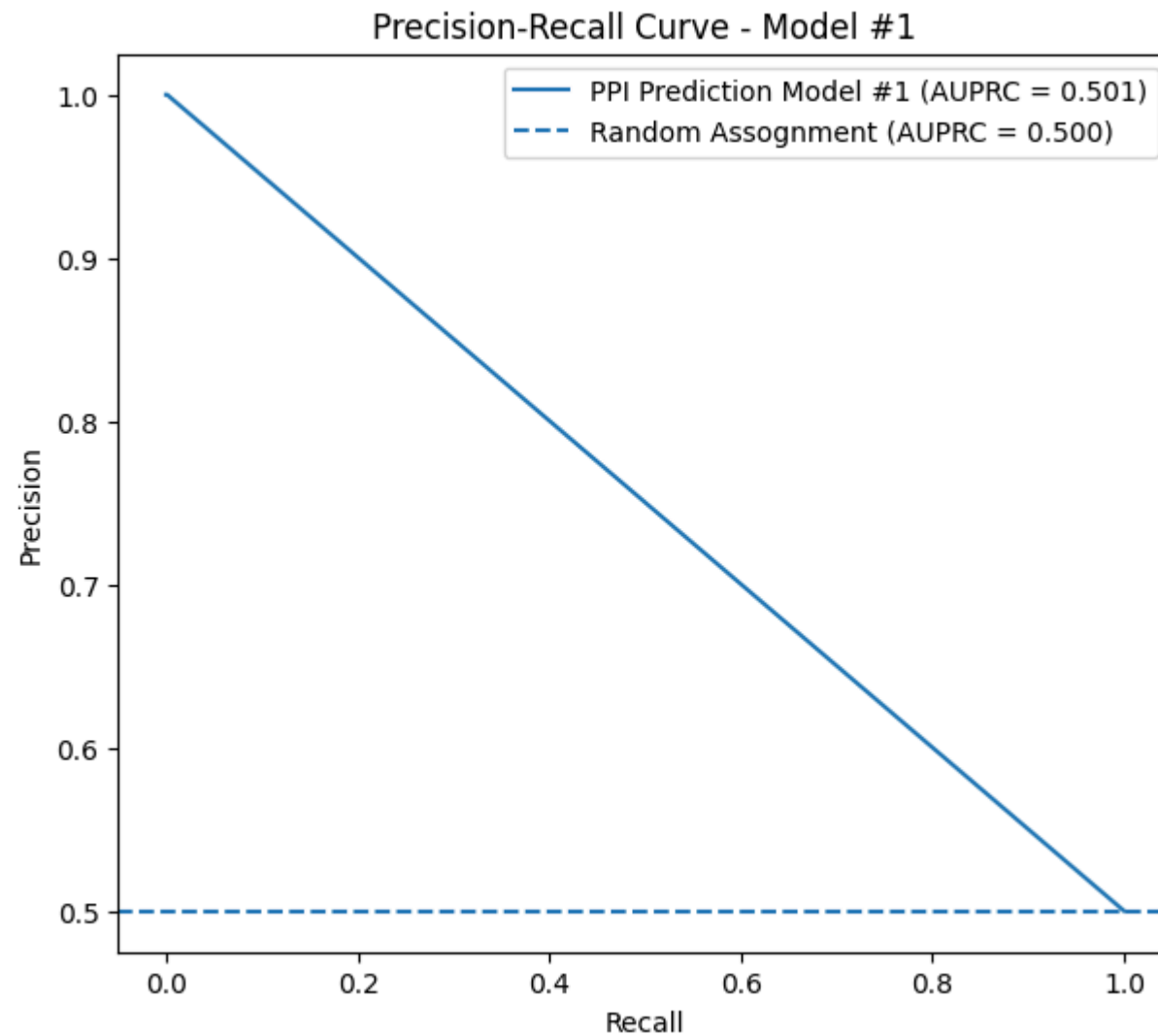
# Plot
```

```
plt.figure(figsize=(7, 6))

plt.plot(
    recall,
    precision,
    label=f'PPI Prediction Model #1 (AUPRC = {auprc:.3f})'
)

# Baseline for a balanced test set
baseline = y_test.mean()
plt.axhline(
    baseline,
    linestyle='--',
    label=f'Random Assognment (AUPRC = {baseline:.3f})'
)

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve - Model #1')
plt.legend()
plt.show()
```



```
In [46]: thresholds = np.arange(0.01, 1.00, 0.01)

f1_scores = []

for threshold in thresholds:
    y_pred1 = (y_prob1 >= threshold).astype(int)
    f1_scores.append(f1_score(y_test, y_pred1))
```

```
best_idx = np.argmax(f1_scores)

best_threshold = thresholds[best_idx]
best_f1_1 = f1_scores[best_idx]

print("Best threshold:", best_threshold)
print("Best F1:", best_f1_1)
```

Best threshold: 0.01

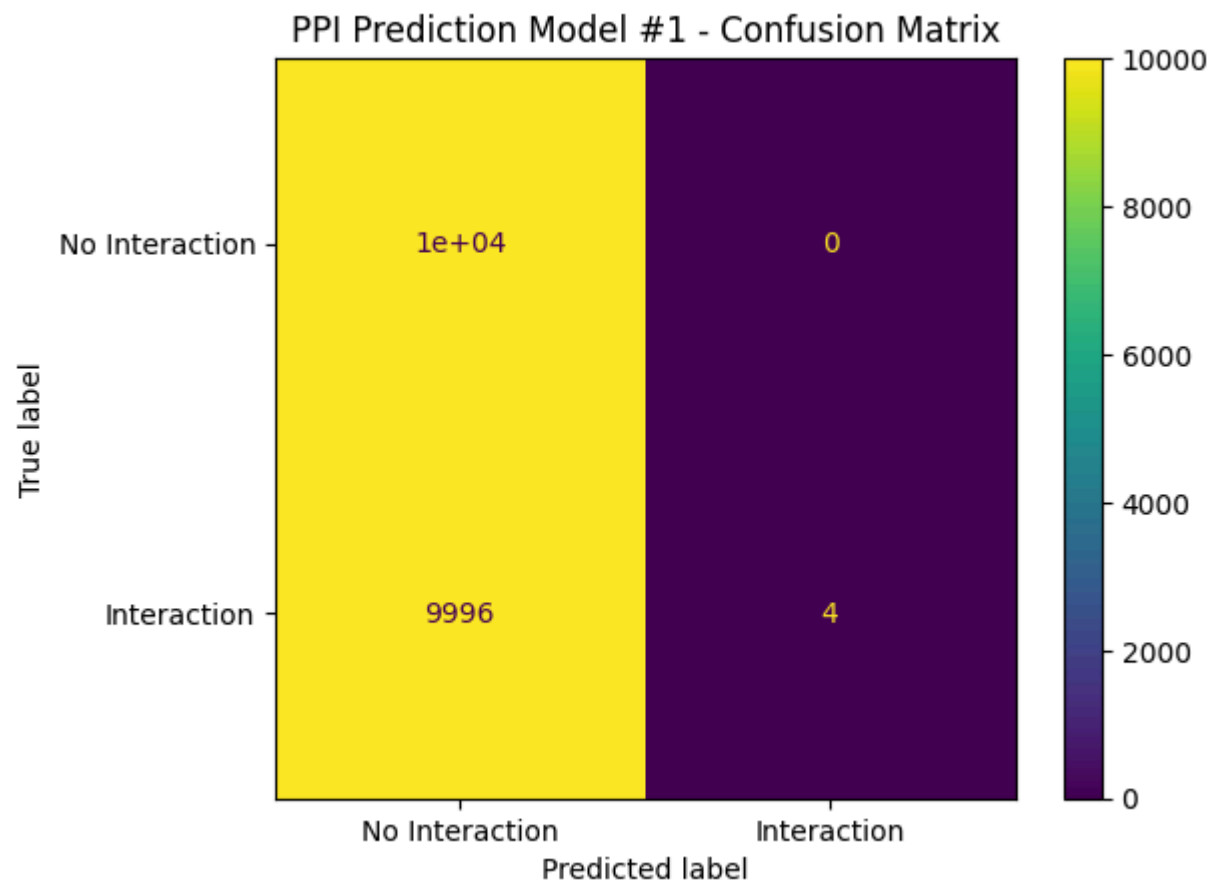
Best F1: 0.0007996801279488205

```
In [47]: # Create a confusion matrix
# Convert probabilities to 0/1 predictions
y_pred1 = (y_prob1 >= 0.5).astype(int)

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred1)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['No Interaction', 'Interaction']
)

disp.plot()
plt.title('PPI Prediction Model #1 - Confusion Matrix')
plt.show()
```



Now we evaluate the model trained with scled embedding vectors

```
In [48]: # Evaluate the model (1280-dimensional embedding, with scaling)
results = model2.evaluate(
    X_test_scaled,
    y_test,
    verbose=0
)

for name, value in zip(model2.metrics_names, results):
    print(f"{name}: {value:.4f}")
```

```
results2 = model2.evaluate(X_test_scaled, y_test, verbose=0, return_dict=True)

print(results2)
```

```
loss: 7033868288.0000
compile_metrics: 0.5968
{'accuracy': 0.596750020980835, 'auc': 0.596750020980835, 'loss': 7033868288.0, 'precision': 0.5744746327400208, 'recall': 0.7462999820709229}
```

```
In [49]: y_prob2 = model2.predict(X_test_scaled).ravel()

roc_auc2 = roc_auc_score(y_test, y_prob2)
APS2 = average_precision_score(y_test, y_prob2)

print('ROC-AUC:', roc_auc2)
print('PR-AUC:', APS2)
```

```
625/625 ————— 1s 1ms/step
ROC-AUC: 0.59675
PR-AUC: 0.5555804210607342
```

```
In [50]: y_pred_labels2 = (y_prob2 >= 0.5).astype(int)

f1_2 = f1_score(y_test, y_pred_labels2)

print('f1 Score:', f1_2)
```

```
f1 Score: 0.649210560654169
```

```
In [51]: # Calculate ROC curve (with scaling)
fpr, tpr, thresholds = roc_curve(y_test, y_prob2)

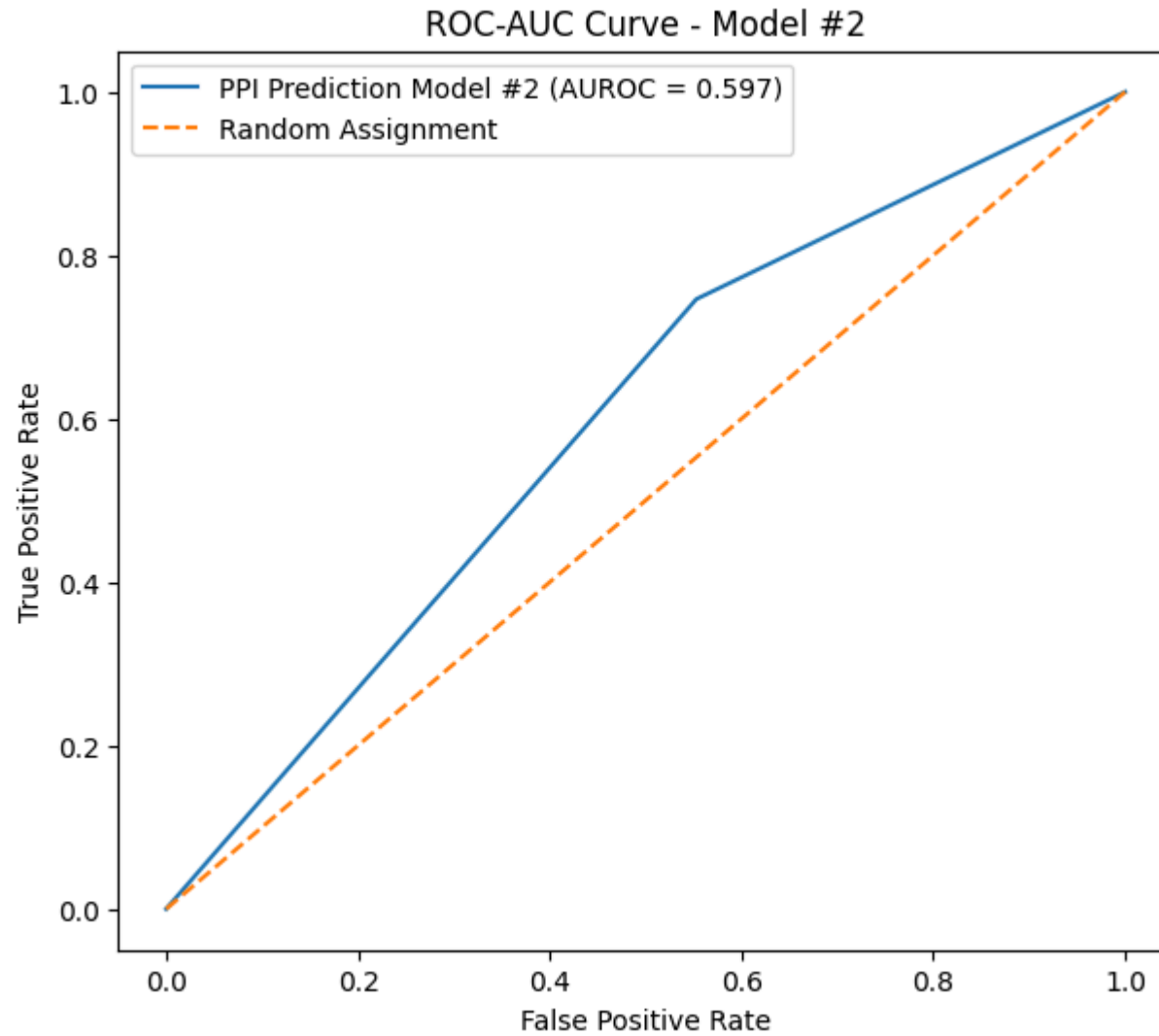
# Calculate AUROC
auc = roc_auc_score(y_test, y_prob2)

# Plot
plt.figure(figsize=(7, 6))

plt.plot(fpr, tpr, label=f'PPI Prediction Model #2 (AUROC = {auc:.3f})')

# Random classifier
plt.plot([0, 1], [0, 1], linestyle='--', label='Random Assignment')
```

```
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC-AUC Curve - Model #2')  
plt.legend()  
plt.show()
```



```
In [52]: # Calculate precision-recall curve (with scaling)
precision, recall, thresholds = precision_recall_curve(y_test, y_prob2)

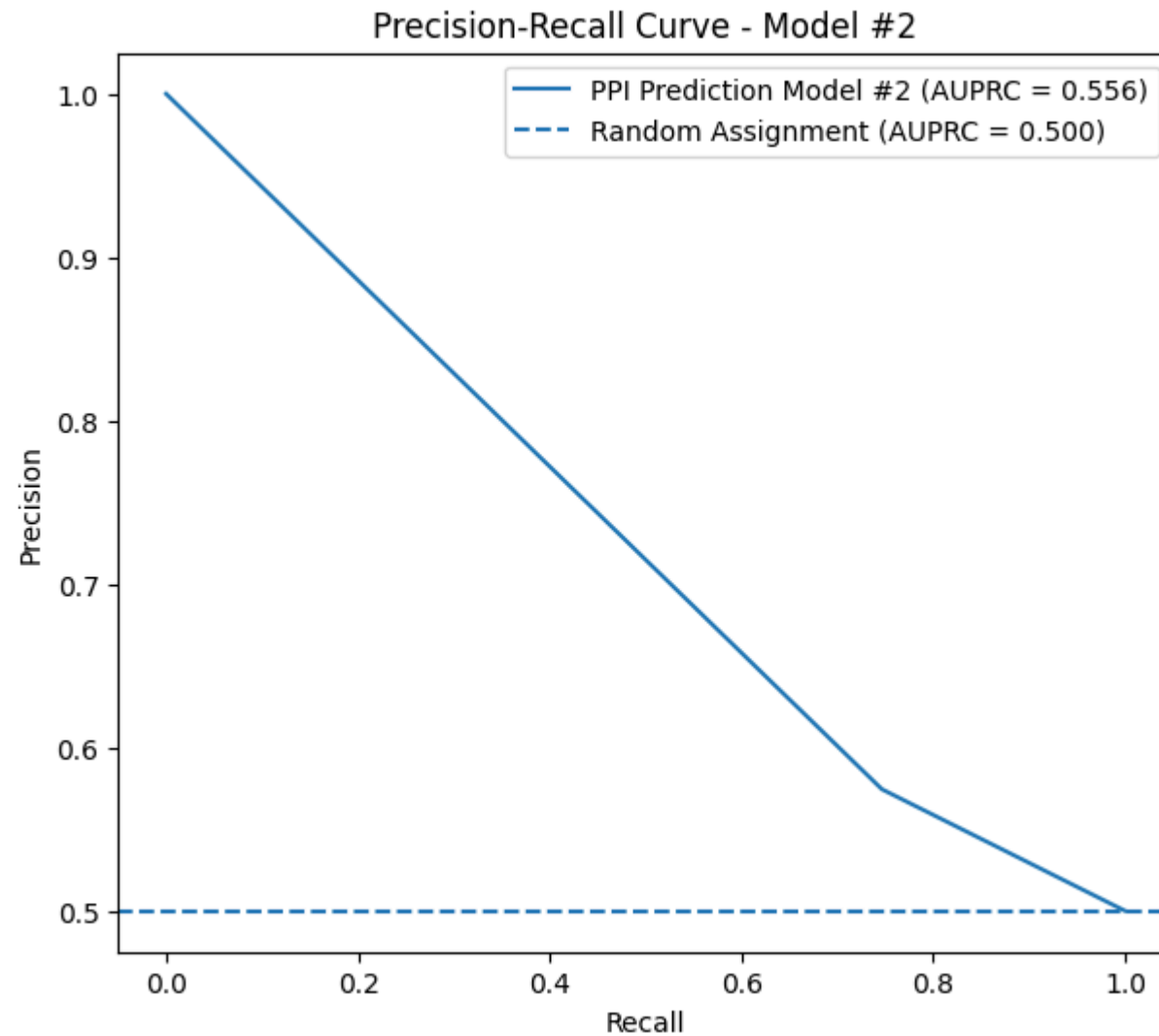
# Calculate AUPRC
auprc = average_precision_score(y_test, y_prob2)

# Plot
plt.figure(figsize=(7, 6))

plt.plot(
    recall,
    precision,
    label=f'PPI Prediction Model #2 (AUPRC = {auprc:.3f})'
)

# Baseline for a balanced test set
baseline = y_test.mean()
plt.axhline(
    baseline,
    linestyle='--',
    label=f'Random Assignment (AUPRC = {baseline:.3f})'
)

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve - Model #2')
plt.legend()
plt.show()
```



```
In [53]: thresholds = np.arange(0.01, 1.00, 0.01)

f1_scores = []

for threshold in thresholds:
    y_pred = (y_prob2 >= threshold).astype(int)
    f1_scores.append(f1_score(y_test, y_pred))
```

```
best_idx = np.argmax(f1_scores)

best_threshold = thresholds[best_idx]
best_f1_2 = f1_scores[best_idx]

print('Best threshold:', best_threshold)
print('Best F1:', best_f1_2)
```

Best threshold: 0.01

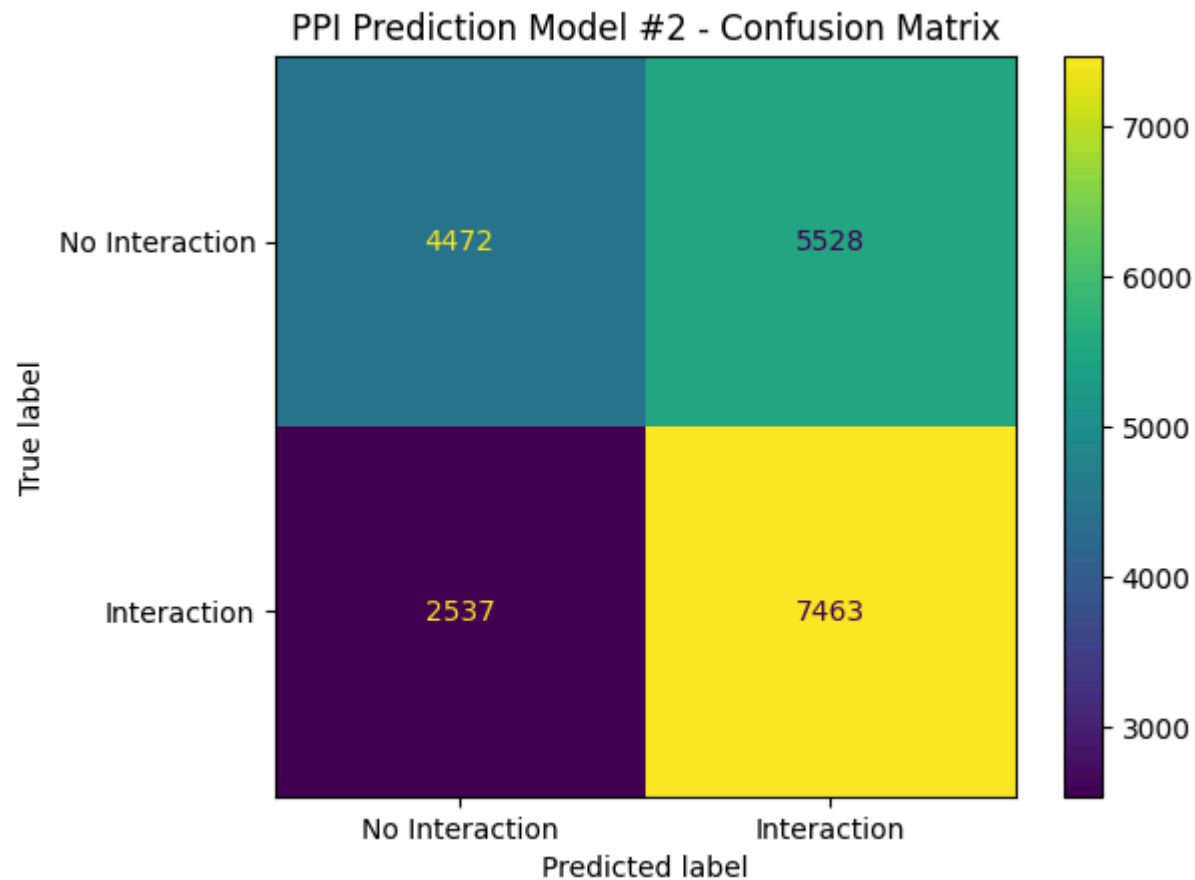
Best F1: 0.649210560654169

```
In [54]: # Create a confusion matrix
# Convert probabilities to 0/1 predictions
y_pred = (y_prob2 >= 0.5).astype(int)

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['No Interaction', 'Interaction']
)

disp.plot()
plt.title('PPI Prediction Model #2 - Confusion Matrix')
plt.show()
```



Finally, we evaluate the model trained using the smaller embeddings (and scaled data)

```
In [55]: # Evaluate the model (640-dimensional embedding, with scaling)
results_small = model_small.evaluate(
    X_test_scaled_small,
    y_test_small,
    verbose=0
)

for name, value in zip(model_small.metrics_names, results_small):
    print(f"{name}: {value:.4f}")
```

```
results_small = model_small.evaluate(X_test_scaled_small, y_test_small, verbose=0, return_dict=True)

print(results_small)
```

loss: 98013088.0000

compile_metrics: 0.5871

```
{'accuracy': 0.587149977684021, 'auc': 0.587149977684021, 'loss': 98013088.0, 'precision': 0.5616467595100403, 'recall': 0.7940000295639038}
```

```
In [56]: y_prob_small = model_small.predict(X_test_scaled_small).ravel()
```

```
roc_auc_small = roc_auc_score(y_test, y_prob_small)
APS_small = average_precision_score(y_test, y_prob_small)
```

```
print('ROC-AUC:', roc_auc_small)
print('PR-AUC:', APS_small)
```

625/625 ————— 1s 1ms/step

ROC-AUC: 0.58715

PR-AUC: 0.5489475136167503

```
In [57]: y_pred_labels_small = (y_prob_small >= 0.5).astype(int)
```

```
f1_small = f1_score(y_test_small, y_pred_labels_small)
```

```
print('f1 Score:', f1_small)
```

f1 Score: 0.6579110908563616

```
In [58]: # Calculate ROC curve (with scaling)
fpr, tpr, thresholds = roc_curve(y_test_small, y_prob_small)
```

```
# Calculate AUROC
```

```
auc = roc_auc_score(y_test, y_prob_small)
```

```
# Plot
```

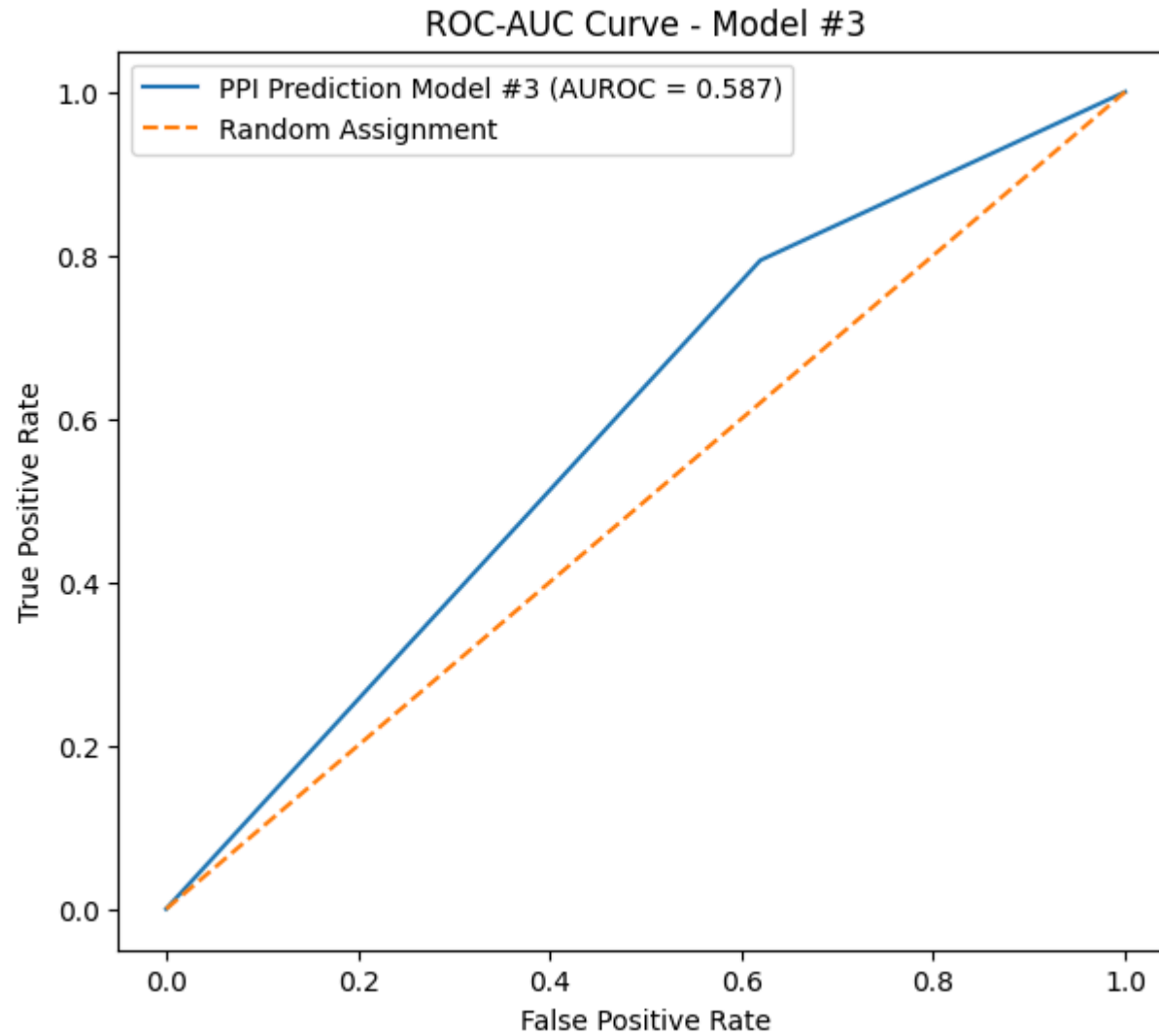
```
plt.figure(figsize=(7, 6))
```

```
plt.plot(fpr, tpr, label=f'PPI Prediction Model #3 (AUROC = {auc:.3f})')
```

```
# Random classifier
```

```
plt.plot([0, 1], [0, 1], linestyle='--', label='Random Assignment')
```

```
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC-AUC Curve - Model #3')  
plt.legend()  
plt.show()
```



```
In [59]: # Calculate precision-recall curve (with scaling)
precision, recall, thresholds = precision_recall_curve(y_test, y_prob_small)

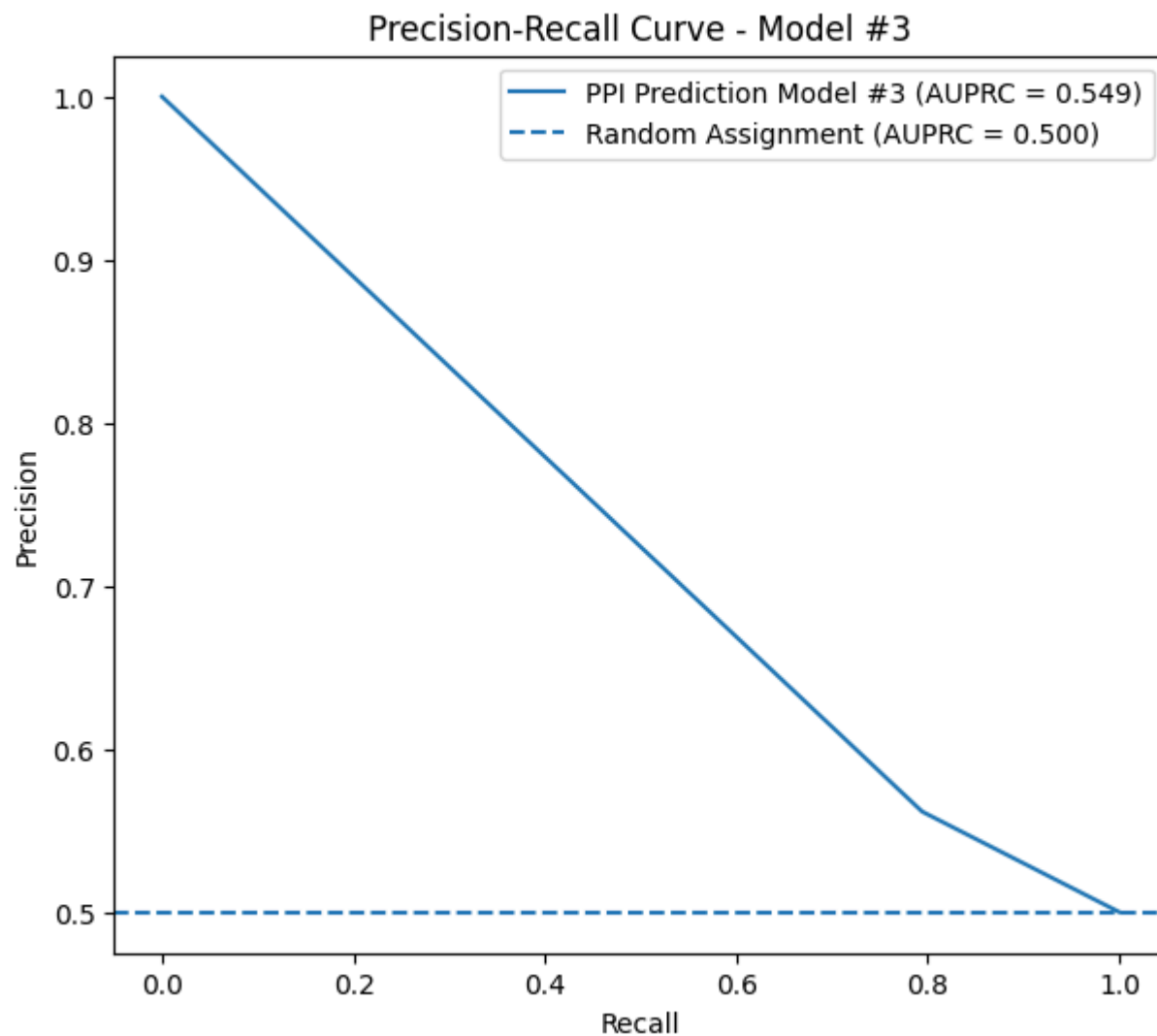
# Calculate AUPRC
auprc = average_precision_score(y_test, y_prob_small)

# Plot
plt.figure(figsize=(7, 6))

plt.plot(
    recall,
    precision,
    label=f'PPI Prediction Model #3 (AUPRC = {auprc:.3f})'
)

# Baseline for a balanced test set
baseline = y_test.mean()
plt.axhline(
    baseline,
    linestyle='--',
    label=f'Random Assignment (AUPRC = {baseline:.3f})'
)

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve - Model #3')
plt.legend()
plt.show()
```



```
In [60]: thresholds = np.arange(0.01, 1.00, 0.01)

f1_scores = []

for threshold in thresholds:
    y_pred_small = (y_prob_small >= threshold).astype(int)
    f1_scores.append(f1_score(y_test_small, y_pred_small))
```

```
best_idx = np.argmax(f1_scores)

best_threshold = thresholds[best_idx]
best_f1_small = f1_scores[best_idx]

print('Best threshold:', best_threshold)
print('Best F1:', best_f1_small)
```

Best threshold: 0.01

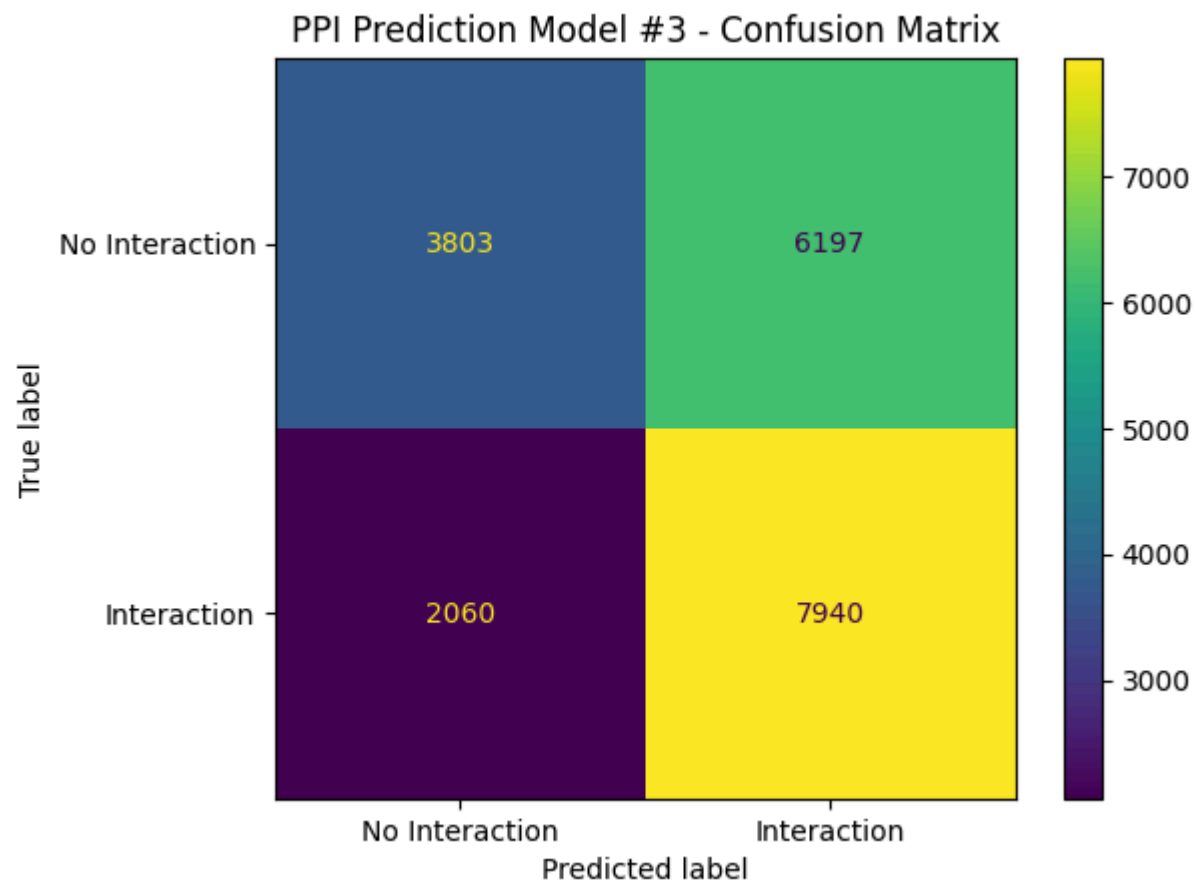
Best F1: 0.6579110908563616

```
In [61]: # Create a confusion matrix
# Convert probabilities to 0/1 predictions
y_pred_small = (y_prob_small >= 0.5).astype(int)

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred_small)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['No Interaction', 'Interaction']
)

disp.plot()
plt.title('PPI Prediction Model #3 - Confusion Matrix')
plt.show()
```



Now we compare all 3 models:

```
In [62]: # Compile the results dictionaries and display in a table
results_small.pop('loss', None)
results1.pop('loss', None)
results2.pop('loss', None)

results_small.pop('auc', None)
results1.pop('auc', None)
results2.pop('auc', None)

results_small['f1 Score'] = f1_small
```

```

results1['f1 Score'] = f1_1
results2['f1 Score'] = f1_2

results_small['ROC-AUC'] = roc_auc_small
results1['ROC-AUC'] = roc_auc1
results2['ROC-AUC'] = roc_auc2

results_small['PR-AUC'] = APS_small
results1['PR-AUC'] = APS1
results2['PR-AUC'] = APS2

comparison = pd.DataFrame({
    'ESM2-640 (w/ Scaling)': results_small,
    'ESM2-1280 (No Scaling)': results1,
    'ESM2-1280 (w/ Scaling)': results2
})

display(comparison)

```

	ESM2-640 (w/ Scaling)	ESM2-1280 (No Scaling)	ESM2-1280 (w/ Scaling)
accuracy	0.587150	0.5002	0.596750
precision	0.561647	1.0000	0.574475
recall	0.794000	0.0004	0.746300
f1 Score	0.657911	0.0008	0.649211
ROC-AUC	0.587150	0.5006	0.596750
PR-AUC	0.548948	0.5006	0.555580

In []: